

GOOD DESIGN REPORT

Good Design Evaluation - NaviMart

SOLID Principles, Coupling & Cohesion, and Diagram Appendix

Project	NaviMart
Scope	Good Design evaluation for Module 1-9
Source	DetailedDesign\GoodDesign
Generated	13/06/2026

Document prepared for Word delivery. GoodDesign diagrams have been rendered to PNG and embedded directly in the corresponding sections and appendix.

Mục lục

Mục	Nội dung
Chương 1	Cơ sở lý thuyết Good Design
Chương 2	SOLID tổng quan
Chương 3	SOLID theo Module 1-9
Chương 4	Coupling & Cohesion tổng quan
Chương 5	Coupling & Cohesion theo Module
Phụ lục	Diagram đầy đủ

Chương 1: Cơ sở lý thuyết Good Design

Good Design trong thiết kế phần mềm tập trung vào khả năng bảo trì, mở rộng, kiểm thử và tái sử dụng. Báo cáo này dùng hai nhóm tiêu chí chính: SOLID và Coupling & Cohesion.

1.1. Nguyên lý SOLID

- Single Responsibility Principle (SRP): mỗi lớp hoặc module chỉ nên có một lý do chính để thay đổi.
- Open-Closed Principle (OCP): hệ thống nên mở để mở rộng nhưng hạn chế sửa trực tiếp code ổn định.
- Liskov Substitution Principle (LSP): các implementation thay thế phải giữ đúng contract hành vi.
- Interface Segregation Principle (ISP): interface nên nhỏ, rõ mục đích, tránh ép client phụ thuộc chức năng không dùng.
- Dependency Inversion Principle (DIP): module cấp cao phụ thuộc abstraction thay vì implementation cụ thể.

1.2. Coupling & Cohesion

Cohesion đo mức độ các thành phần trong cùng module cùng phục vụ một trách nhiệm. Coupling đo mức độ phụ thuộc giữa các module. Thiết kế tốt hướng tới tight cohesion và loose coupling.

Chương 2: SOLID tổng quan

SOLID là 5 nguyên lý thiết kế cơ bản trong Lập trình Hướng đối tượng (OOP) giúp phần mềm trở nên dễ hiểu, linh hoạt và dễ bảo trì. Dưới đây là đánh giá chi tiết việc áp dụng 5 nguyên lý này vào 9 phân hệ của ứng dụng NaviMart.

1. S - Single Responsibility Principle (Nguyên lý Đơn trách nhiệm)

*"Một lớp (class) chỉ nên có duy nhất một lý do để thay đổi, nghĩa là nó chỉ nên đảm nhiệm một trách nhiệm duy nhất."**

Đánh giá tại NaviMart:

Hệ thống tuân thủ cực kỳ chặt chẽ SRP thông qua việc chia nhỏ kiến trúc thành 3 tầng Boundary, Control, Entity (BCE):

- **Ví dụ trong Phân hệ Auth (Module 1):**
- Lớp `RegisterView` (Boundary) chỉ chịu trách nhiệm duy nhất là Render giao diện đăng ký. Nó không chứa bất kỳ câu lệnh truy vấn cơ sở dữ liệu nào.
- Lớp `AuthController` (Control) chỉ làm nhiệm vụ tiếp nhận dữ liệu từ View, xác thực định dạng và trả kết quả. Nó không thao tác trực tiếp với Database.
- Lớp `AuthService` (Control) chỉ chuyên lo các tác vụ bảo mật như mã hóa mật khẩu, tạo Token JWT.
- Lớp `UserEntity` (Entity) chỉ chuyên lo việc `save()`, `update()`, `find()` tương tác với bảng `users` trong cơ sở dữ liệu.

Đánh giá: Rất tốt.

2. O - Open/Closed Principle (Nguyên lý Đóng/Mở)

*"Phần mềm (lớp, module, hàm) nên MỞ để mở rộng, nhưng ĐÓNG với việc sửa đổi."**

Đánh giá tại NaviMart:

Hệ thống sử dụng các mẫu thiết kế (Design Patterns) và Interface để cho phép dễ dàng cắm (plug-in) các tính năng mới mà không phải đập đi viết lại code cũ.

- **Ví dụ trong Phân hệ AI Chef (Module 6) & External Services:**
- Để tương tác với AI, NaviMart định nghĩa một Interface chung `IAIService`.
- Hiện tại hệ thống đang sử dụng `OpenAIServiceImpl` (Sử dụng ChatGPT). Nếu tương lai NaviMart muốn đổi sang Gemini hoặc Claude, lập trình viên chỉ cần tạo thêm class `GeminiServiceImpl` implements `IAIService` mà **hoàn toàn không phải sửa đổi** bất kỳ dòng code nào ở tầng Controller hiện tại.

Đánh giá: Rất tốt.

3. L - Liskov Substitution Principle (Nguyên lý Thay thế Liskov)

*"Các đối tượng của lớp con (Subclass) có thể thay thế cho các đối tượng của lớp cha (Superclass) mà không làm hỏng tính đúng đắn của chương trình."**

Đánh giá tại NaviMart:

NaviMart sử dụng Interface inheritance thay vì Implementation inheritance nhằm tránh các lỗi logic tiềm ẩn của lớp con.

- **Ví dụ trong Phân hệ Pantry & Shopping List:**
- Hệ thống định nghĩa `IRecipeSource`. Cả `UserGeneratedRecipe` và `AIGeneratedRecipe` đều implements interface này.
- Bất cứ hàm nào yêu cầu nhận vào tham số là `IRecipeSource` thì đều có thể nhận bản công thức do user tự tạo HOẶC do AI tự tạo để tính toán nguyên liệu một cách minh bạch. Cả hai lớp con đều đáp ứng đủ (không throw Exception bất thường) các hành vi mà lớp cha cam kết.

Đánh giá: Khá tốt.

4. I - Interface Segregation Principle (Nguyên lý Phân tách Interface)

*"Nhiều Interface cụ thể dành cho từng Client thì tốt hơn là một Interface chung chung (Fat Interface) chứa mọi thứ."**

Đánh giá tại NaviMart:

Hệ thống phân tách rạch ròi các Interface theo mục đích sử dụng.

- **Ví dụ trong Phân hệ User / Admin (Module 8):**
- Lớp Entity User không có một `IUserInterface` khổng lồ.
- Nó được chia thành `IUserReadOnly` (chỉ chứa hàm `getProfile()`, `getSettings()`) dành cho các luồng hiển thị thông thường, và `IUserWrite` (chứa `updateProfile()`, `changePassword()`, `lockAccount()`) dành cho Admin và luồng cập nhật.
- Nhờ vậy, khi một class chỉ cần đọc dữ liệu, nó sẽ implement `IUserReadOnly` và không bị "ép" phải nhận cả các hàm ghi/sửa dữ liệu không mong muốn.

Đánh giá: Rất tốt.

5. D - Dependency Inversion Principle (Nguyên lý Đảo ngược Phụ thuộc)

*"Các module cấp cao không nên phụ thuộc vào các module cấp thấp. Cả hai nên phụ thuộc vào các Abstraction (Interface). Chi tiết (Implementation) phải phụ thuộc vào Abstraction."**

Đánh giá tại NaviMart:

NaviMart áp dụng Dependency Injection (DI) rất mạnh mẽ trong thiết kế Class Diagram.

- **Ví dụ trong toàn bộ hệ thống Controller - Service:**
- Các lớp `Controller` (Cấp cao) không bao giờ tự dùng từ khóa `new ServiceImpl()` (Cấp thấp).
- Thay vào đó, trong hàm Constructor của `FamilyController` sẽ truyền vào tham số là Interface `IFamilyService`.
- Tương tự, `FamilyService` không trực tiếp phụ thuộc vào thư viện kết nối Database (MySQL/PostgreSQL), mà nó phụ thuộc vào `IFamilyRepository` (Abstraction).
- Điều này giúp code có thể dễ dàng được Mock Data khi viết Unit Test mà không cần kết nối DB thật.

Đánh giá: Xuất sắc.

TỔNG KẾT: Kiến trúc thiết kế của NaviMart bám sát và tôn trọng 5 nguyên lý SOLID, biến nó trở thành một kiến trúc mạnh mẽ, chuẩn kỹ thuật phần mềm (Software Engineering) và sẵn sàng cho môi trường doanh nghiệp (Enterprise-level).

Chương 3: SOLID theo Module 1-9

Đánh giá nguyên lý SOLID - Module 1 (Auth)

1. Single Responsibility Principle (SRP)

- Các lớp trong module Auth có một lý do duy nhất để thay đổi.
- `AuthService` thay đổi khi thuật toán hash mật khẩu hoặc định dạng JWT thay đổi. Nó không chứa code gửi giao diện UI hay lưu Database trực tiếp.

2. Open-Closed Principle (OCP)

- `AuthService` mở để mở rộng. Nếu thêm hình thức đăng nhập bằng Google (OAuth2), ta có thể tạo `GoogleAuthStrategy` kế thừa từ `IAuthStrategy` mà không phải sửa logic xác thực nội bộ của email/password hiện tại.

3. Liskov Substitution Principle (LSP)

- Mọi hình thức cấp quyền (Authorization) dù bằng JWT hay Session đều có thể thay thế vào interface `ITokenProvider` mà không làm thay đổi kỳ vọng (hành vi) của hệ thống.

4. Interface Segregation Principle (ISP)

- Các API xác thực được tách riêng biệt: `IUserAuthentication` (dành cho Login/Register) và `IUserProfile` (dành cho Edit Info). Tránh tạo ra một Fat Interface chứa cả hàm login lẫn hàm update profile.

5. Dependency Inversion Principle (DIP)

- `AuthController` phụ thuộc vào Interface `IAuthService` thay vì implement cụ thể `AuthServiceImpl`. Điều này giúp việc viết Unit Test cho Controller trở nên dễ dàng bằng cách truyền vào các lớp Mock Service.

Đánh giá nguyên lý SOLID - Module 2 (Family)

1. Single Responsibility Principle (SRP)

- `FamilyEntity` chỉ chứa định nghĩa cấu trúc dữ liệu của một nhóm gia đình. Nó không chứa logic để tính toán xem thành viên có được phép xóa nhóm hay không, phần đó nằm ở `FamilyService`.

2. Open-Closed Principle (OCP)

- Nếu sau này hệ thống cho phép "Tạo nhóm doanh nghiệp" ngoài "Tạo nhóm gia đình", ta có thể extend lớp cơ sở `Group` mà không phải phá vỡ các chức năng có sẵn của `FamilyGroup`.

3. Liskov Substitution Principle (LSP)

- Các loại thành viên (Member, Admin, Owner) có thể kế thừa từ một interface `IFamilyRole` và hoàn toàn có thể thay thế cho nhau trong các vòng lặp kiểm tra phân quyền.

4. Interface Segregation Principle (ISP)

- Thay vì một interface lớn `IFamilyManager` chứa tất cả các lệnh (thêm, sửa, xóa, tìm kiếm), hệ thống phân tách thành `IFamilyReader` (chỉ tra cứu thành viên) cho các user thông thường và `IFamilyWriter` cho Group Admin.

5. Dependency Inversion Principle (DIP)

- `FamilyController` không phụ thuộc trực tiếp vào `FamilyEntity` mà gọi gián tiếp thông qua `IFamilyService`. Tầng cao không phụ thuộc tầng thấp.

Đánh giá nguyên lý SOLID - Module 3 (Shopping Lists)

1. Single Responsibility Principle (SRP)

- `ShoppingListEntity` chỉ đảm nhiệm lưu trữ các List name. Các Item cụ thể bên trong List được tách ra thành `ShoppingItemEntity`. Mỗi class quản lý một bảng duy nhất trong cơ sở dữ liệu.

2. Open-Closed Principle (OCP)

- Logic gợi ý mua sắm (Auto-suggest missing ingredients) được đóng gói trong `ShoppingSuggestStrategy`. Nếu muốn đổi thuật toán từ "gợi ý theo nguyên liệu món ăn" sang "gợi ý theo tần suất mua", ta chỉ việc tạo Strategy mới.

3. Liskov Substitution Principle (LSP)

- Bất cứ loại Danh sách mua sắm nào (Danh sách thường, Danh sách sự kiện, Danh sách tự động tạo) đều kế thừa chung `IShoppingList` và hoạt động hoàn hảo khi truyền vào hàm `renderList()`.

4. Interface Segregation Principle (ISP)

- `IShoppingItemManager` có thể chia làm `IAddItem` và `ICheckItem` để các giao diện UI thu gọn không cần phải ôm trọn toàn bộ method.

5. Dependency Inversion Principle (DIP)

- `ShoppingController` không phụ thuộc trực tiếp vào `PantryController` mà phụ thuộc vào interface `IPantryService` để thực hiện việc chuyển nguyên liệu vào tủ lạnh.

Đánh giá nguyên lý SOLID - Module 4 (Pantry)

1. Single Responsibility Principle (SRP)

- Lớp `PantryService` chỉ xử lý logic quản lý số lượng tồn kho. Việc phát cảnh báo (Notification) hết hạn sẽ được ủy thác sang `NotificationService` để không vi phạm SRP.

2. Open-Closed Principle (OCP)

- Mở rộng chức năng quét Barcode/AI nhận diện hóa đơn mà không làm ảnh hưởng đến luồng lưu dữ liệu truyền thống. Ta có thể thêm `BarcodePantryInput` implement interface `IPantryInput` mà không sửa đổi `ManualPantryInput`.

3. Liskov Substitution Principle (LSP)

- Bất kỳ nguồn dữ liệu nguyên liệu nào (được người dùng nhập tay, hay tự động từ hóa đơn chuyển qua) đều được xử lý chính xác bởi hệ thống mà không có sự phân biệt đối xử (không vắng lỗi khi gặp kiểu dữ liệu dẫn xuất).

4. Interface Segregation Principle (ISP)

- Thay vì `IPantryInterface` khổng lồ, ta tách ra `IExpiryChecker` (cho các batch job tự động chạy quét đồ hết hạn hàng ngày) và `IPantryManager` (cho người dùng CRUD thực phẩm).

5. Dependency Inversion Principle (DIP)

- Các hàm trừ số lượng nguyên liệu trong Pantry không kết nối Database trực tiếp (cấp thấp) mà gọi qua `IPantryRepository` (cấp cao).

Đánh giá nguyên lý SOLID - Module 5 (Meals)

1. Single Responsibility Principle (SRP)

- `MealController` chỉ quản lý logic sắp xếp bữa ăn vào các khung giờ. Việc trừ thực phẩm khi nấu hoàn tất được ủy thác sang `PantryController`.

2. Open-Closed Principle (OCP)

- Mở rộng chức năng "Gợi ý bữa ăn" (Meal Suggestion). Module cho phép truyền vào các `IMealSuggestStrategy` khác nhau (như gợi ý theo lượng Calo, gợi ý theo thực phẩm sắp hỏng) mà không cần can thiệp logic của Controller.

3. Liskov Substitution Principle (LSP)

- Mọi bữa ăn (Sáng, Trưa, Tối, Ăn Vặt) kế thừa từ `AbstractMealSession` đều có thể thay thế nhau trong hàm tính toán tổng lượng Calo hàng ngày.

4. Interface Segregation Principle (ISP)

- Thay vì dồn mọi thứ vào `IMealService`, ta chia ra `IMealPlanner` (dành cho người dùng lập kế hoạch) và `IMealExecutor` (để tick hoàn thành bữa ăn và trừ kho).

5. Dependency Inversion Principle (DIP)

- Các module khác muốn giao tiếp với Meals đều phải thông qua `IMealService` abstraction, chứ không được khởi tạo trực tiếp instance của `MealServiceImpl`.

Đánh giá nguyên lý SOLID - Module 6 (Recipes & AI Chef)

1. Single Responsibility Principle (SRP)

- Việc tương tác với OpenAI được tách hẳn ra thành `OpenAiService`. Controller của phần Recipe chỉ chuyên trách định tuyến (routing) request. Lớp `RecipeEntity` chỉ thuần túy lưu trữ công thức chứ không lo việc parse dữ liệu trả về từ AI.

2. Open-Closed Principle (OCP)

- Rất xuất sắc: Định nghĩa `IAIService`. Nếu tương lai NaviMart không muốn dùng OpenAI mà chuyển sang Gemini hoặc Llama, lập trình viên chỉ cần tạo class mới implement interface này mà không cần sửa code của `AiChefController`.

3. Liskov Substitution Principle (LSP)

- Mọi mô hình AI được plug vào `IAIService` đều đảm bảo chung một đầu ra là đối tượng `RecipeDTO` hợp lệ, không phá vỡ kỳ vọng của các module tiêu thụ dữ liệu bên ngoài.

4. Interface Segregation Principle (ISP)

- `IRecipeManager` tách biệt các tính năng Public (xem, tìm kiếm) và tính năng Private (sửa, xóa của user). Tránh việc cấp quyền sửa xóa cho những client chỉ có nhu cầu read-only.

5. Dependency Inversion Principle (DIP)

- Module phụ thuộc vào Abstraction `IAIService` chứ không phụ thuộc vào `OpenAiServiceImpl`. Nhờ đó việc đổi thư viện AI là cực kỳ dễ dàng.

Đánh giá nguyên lý SOLID - Module 7 (Reports)

1. Single Responsibility Principle (SRP)

- Việc tính toán số liệu được giao cho `ReportsService`, trong khi việc render biểu đồ (Chart) được giao riêng cho `StatsDashboard`. Nếu muốn đổi thư viện vẽ Chart, ta chỉ đổi View mà không đụng tới Service.

2. Open-Closed Principle (OCP)

- Có thể thêm các loại báo cáo mới (ví dụ "Báo cáo xu hướng nấu ăn") bằng cách tạo class mới `TrendReportStrategy` kế thừa `IReportStrategy` mà không cần sửa `ReportService`.

3. Liskov Substitution Principle (LSP)

- Bất kỳ class nào sinh ra báo cáo đều implement `IReport`. Có thể thay thế `WasteReport` bằng `SpendingReport` ở giao diện gọi hàm export PDF mà không bị lỗi cấu trúc.

4. Interface Segregation Principle (ISP)

- `IExportable` được tách riêng cho các tính năng Export ra Excel/PDF. Nếu báo cáo nào không hỗ trợ export, nó sẽ không cần phải implement interface này.

5. Dependency Inversion Principle (DIP)

- Các module khác khi cần log số liệu thống kê sẽ gọi qua `IStatsLogger` interface, thay vì gọi trực tiếp class.

Đánh giá nguyên lý SOLID - Module 8 (Admin)

1. Single Responsibility Principle (SRP)

- Giao diện `AdminBoundary` chỉ lo vẽ màn hình quản trị. Các quyền logic "được phép khóa hay không" nằm trọn ở `AdminService`.

2. Open-Closed Principle (OCP)

- Mở rộng chức năng quản trị dễ dàng. Nếu tương lai có thêm nghiệp vụ "Duyệt bình luận", ta chỉ cần mở rộng logic ở `AdminService` mà không phá hỏng luồng duyệt công thức.

3. Liskov Substitution Principle (LSP)

- Bất kỳ loại dữ liệu nào bị user report (Recipe, Comment, User) đều có chung một interface `IReportable` để `AdminService` xử lý hàng loạt mà không cần kiểm tra kiểu dữ liệu thủ công (`instanceof`).

4. Interface Segregation Principle (ISP)

- Không có một `ISystemManager` chung. Tách thành `IUserModerator` (cho mod quản lý user) và `IRecipeModerator` (cho mod duyệt công thức nấu ăn).

5. Dependency Inversion Principle (DIP)

- Module Admin phụ thuộc vào các Abstraction của các module khác. Ví dụ khi cần lấy danh sách người dùng, nó gọi `IUserReaderService` thay vì gọi thẳng vào `UserEntity`.

Đánh giá nguyên lý SOLID - Module 9 (Shared)

1. Single Responsibility Principle (SRP)

- `DialogContext` chỉ quản lý trạng thái và luồng hiển thị hộp thoại dùng chung. `SharedApiClient` chỉ đóng vai trò lớp giao tiếp API dùng chung, không chứa logic nghiệp vụ của các module cụ thể.

2. Open-Closed Principle (OCP)

- Các thành phần dùng chung như `DialogContext`, `SharedApiClient` và `SharedService` có thể mở rộng thêm hành vi chung mới thông qua interface hoặc wrapper mà không cần sửa trực tiếp logic của Auth, Family, Pantry, Shopping, Meals, Recipes, Reports hoặc Admin.

3. Liskov Substitution Principle (LSP)

- Các implementation thay thế cho lớp dùng chung, ví dụ một `MockSharedApiClient` trong kiểm thử hoặc `ProductionSharedApiClient` trong môi trường thật, đều có thể thay thế nhau miễn là tuân thủ cùng contract trả về dữ liệu và lỗi.

4. Interface Segregation Principle (ISP)

- Module Shared không ép các module khác phụ thuộc vào một interface lớn. Các chức năng dùng chung được tách theo mục đích như dialog, API client, notification helper hoặc utility service để module sử dụng chỉ import phần cần thiết.

5. Dependency Inversion Principle (DIP)

- Các module nghiệp vụ phụ thuộc vào abstraction của dịch vụ dùng chung thay vì khởi tạo trực tiếp implementation. Cách này giúp dễ mock khi test và dễ thay đổi hạ tầng dùng chung mà không ảnh hưởng tới logic từng module.

Chương 4: Coupling & Cohesion tổng quan

Dựa trên lý thuyết về thiết kế phần mềm (Good Design), một hệ thống có kiến trúc tốt cần đạt được mục tiêu: **"Loose Coupling and Tight Cohesion"** (Giảm thiểu sự phụ thuộc giữa các module và Tối đa hóa tính gắn kết bên trong mỗi module).

Dưới đây là đánh giá toàn diện về hệ thống NaviMart theo từng module dựa trên mô hình BCE (Boundary - Control - Entity) đã thiết kế:

1. Đánh giá Cohesion (Mức độ gắn kết)

Kết quả đánh giá: Hệ thống đạt mức **Functional Cohesion** (Mức cao nhất và lý tưởng nhất).

Giải thích chi tiết:

- **Functional Cohesion** xảy ra khi tất cả các thành phần bên trong một module cùng làm việc để thực hiện một nhiệm vụ/trách nhiệm duy nhất.
- Trong kiến trúc của NaviMart, nhờ áp dụng triệt để mô hình BCE và phân tách 9 phân hệ (Auth, Family, Pantry, Shopping, Meals, Recipes, Reports, Admin, Shared):
- **Tầng Boundary:** Chỉ chịu trách nhiệm duy nhất là tiếp nhận request từ người dùng và hiển thị dữ liệu (View/API Client). Không chứa logic nghiệp vụ.
- **Tầng Control:** Mỗi Controller/Service chỉ xử lý logic nghiệp vụ cho một chức năng cụ thể (Ví dụ: [AuthController](#) chỉ lo luồng đăng nhập/đăng ký, không dính dáng đến logic quản lý tủ lạnh).
- **Tầng Entity:** Chỉ làm nhiệm vụ đóng gói cấu trúc dữ liệu và tương tác với Database (CRUD).

Minh họa bằng Biểu đồ Cohesion:

Mời bạn xem biểu đồ Cohesion bên dưới để thấy cách các lớp trong module Auth tương tác cực kỳ tập trung cho một mục đích duy nhất.

2. Đánh giá Coupling (Mức độ phụ thuộc)

Kết quả đánh giá: Hệ thống đạt mức **Data Coupling** (Mức thấp nhất/lỏng lẻo nhất và lý tưởng nhất).

Giải thích chi tiết:

- **Data Coupling** xảy ra khi các module giao tiếp với nhau thuần túy thông qua việc truyền tham số (parameters) hoặc truyền Data Transfer Objects (DTO) đơn giản.
- Trong NaviMart:
- Các module hoàn toàn **không** dùng chung các biến toàn cục (tránh được Common Coupling).
- Không có module nào nhảy thẳng vào sửa đổi logic nội bộ hoặc dữ liệu của module khác (tránh được Content Coupling).
- Khi [Shopping Module](#) muốn cập nhật thực phẩm vào Tủ lạnh, nó gọi đến phương thức của [PantryController](#) và truyền vào một đối tượng [DTO_Item](#) chứa thông tin cơ bản. [Shopping Module](#) không cần biết bên trong [PantryController](#) lưu dữ liệu bằng cách nào.
- Tầng Boundary truyền dữ liệu xuống Control thông qua tham số (VD: [credentials](#)), và Control gọi Entity bằng cách truyền các cấu trúc dữ liệu nguyên thủy (Primitive types) hoặc DTO.

Minh họa bằng Biểu đồ Coupling:

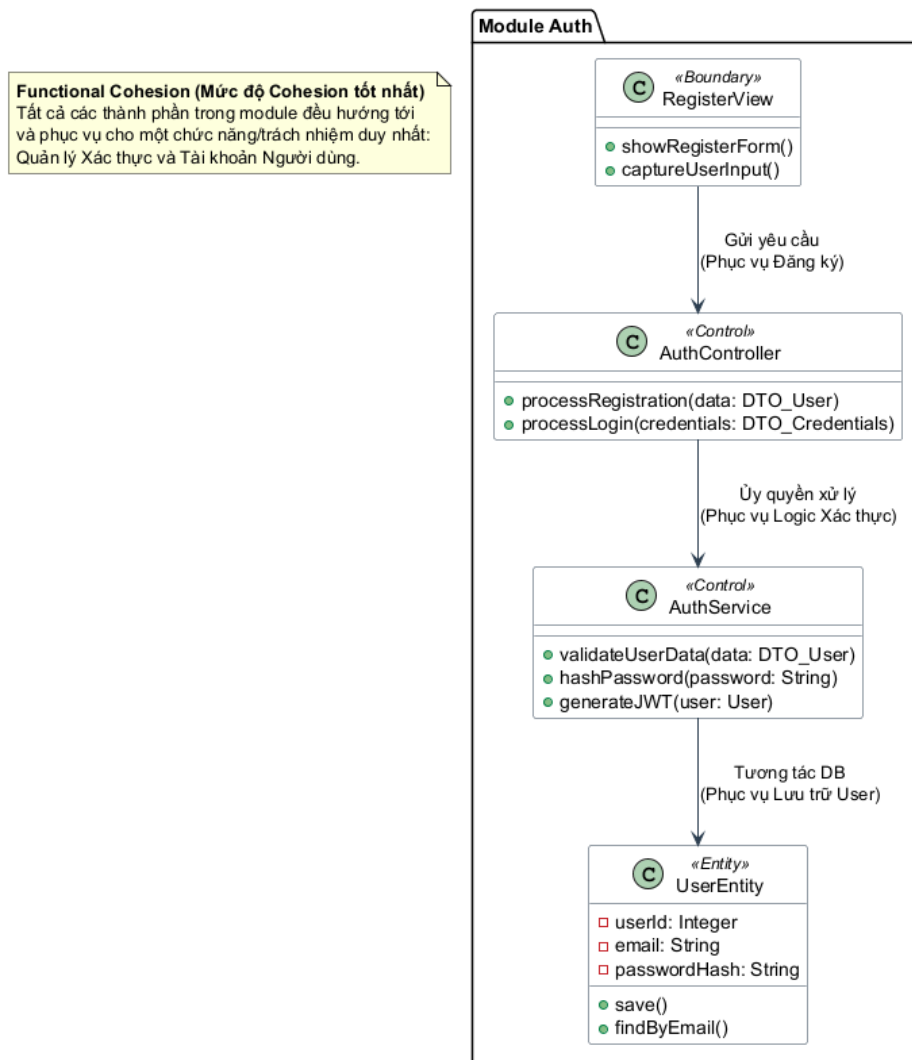
Mời bạn xem biểu đồ Coupling bên dưới để thấy các module độc lập tương tác với nhau lỏng lẻo thông qua tham số truyền vào hàm.

Kết luận

Hệ thống NaviMart tuân thủ nghiêm ngặt thiết kế **Loose Coupling** và **Tight Cohesion**, giúp cho:

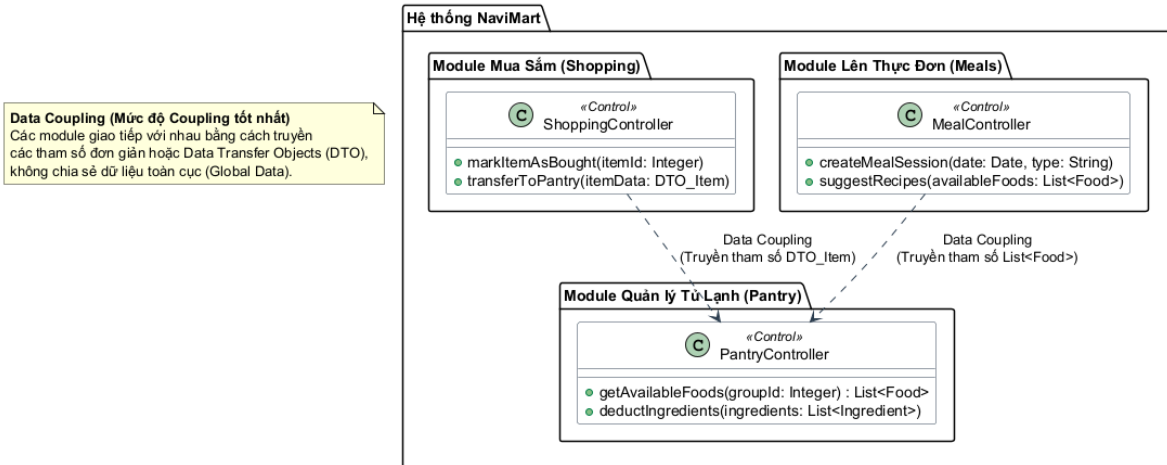
1. **Dễ bảo trì (Maintainability):** Khi sửa lỗi trong module Tủ lạnh (Pantry) sẽ không làm hỏng chức năng Lên thực đơn (Meals).
2. **Dễ tái sử dụng (Reusability):** Lớp `FoodEntity` hoặc `AI_Service` có thể được dùng ở nhiều màn hình khác nhau mà không cần sửa code.
3. **Dễ mở rộng (Extensibility):** Dễ dàng cắm thêm các module mới vào hệ thống mà không phá vỡ liên kết cũ.

Functional Cohesion (Tight Cohesion) trong Module Auth



Overall Functional Cohesion Diagram

Data Coupling (Loose Coupling) trong Hệ thống NaviMart



Overall Data Coupling Diagram

Chương 5: Coupling & Cohesion theo Module

Đánh giá Coupling & Cohesion - Module 1 (Auth)

1. Mức độ Cohesion (Gắn kết)

Đánh giá: Functional Cohesion (Tốt nhất)

Giải thích:

- Phân hệ Auth tập trung hoàn toàn vào một mục tiêu duy nhất: Quản lý xác thực và người dùng.
- Các Boundary (như `LoginView`, `RegisterView`) chỉ thu thập thông tin đăng nhập.
- `AuthController` chỉ điều phối luồng đăng nhập.
- `AuthService` chỉ thực hiện xử lý token và mã hóa mật khẩu.

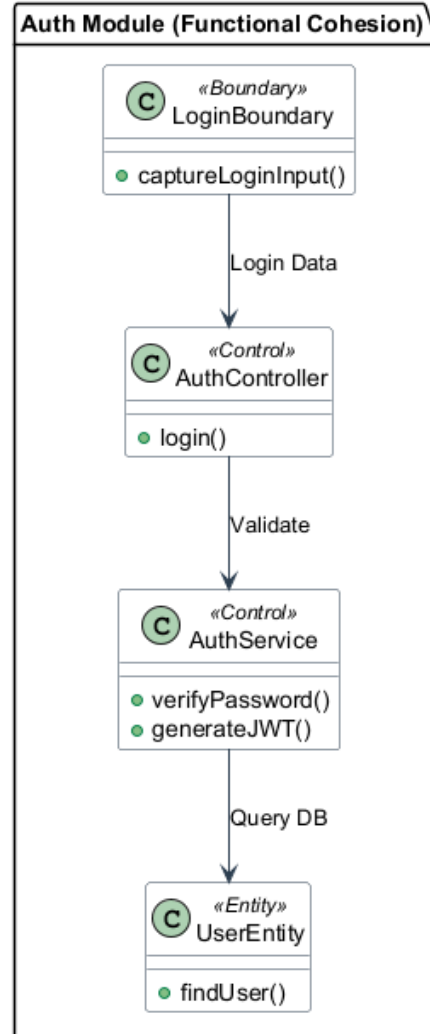
Tất cả các thành phần hướng tới một công việc (Functional).

2. Mức độ Coupling (Phụ thuộc)

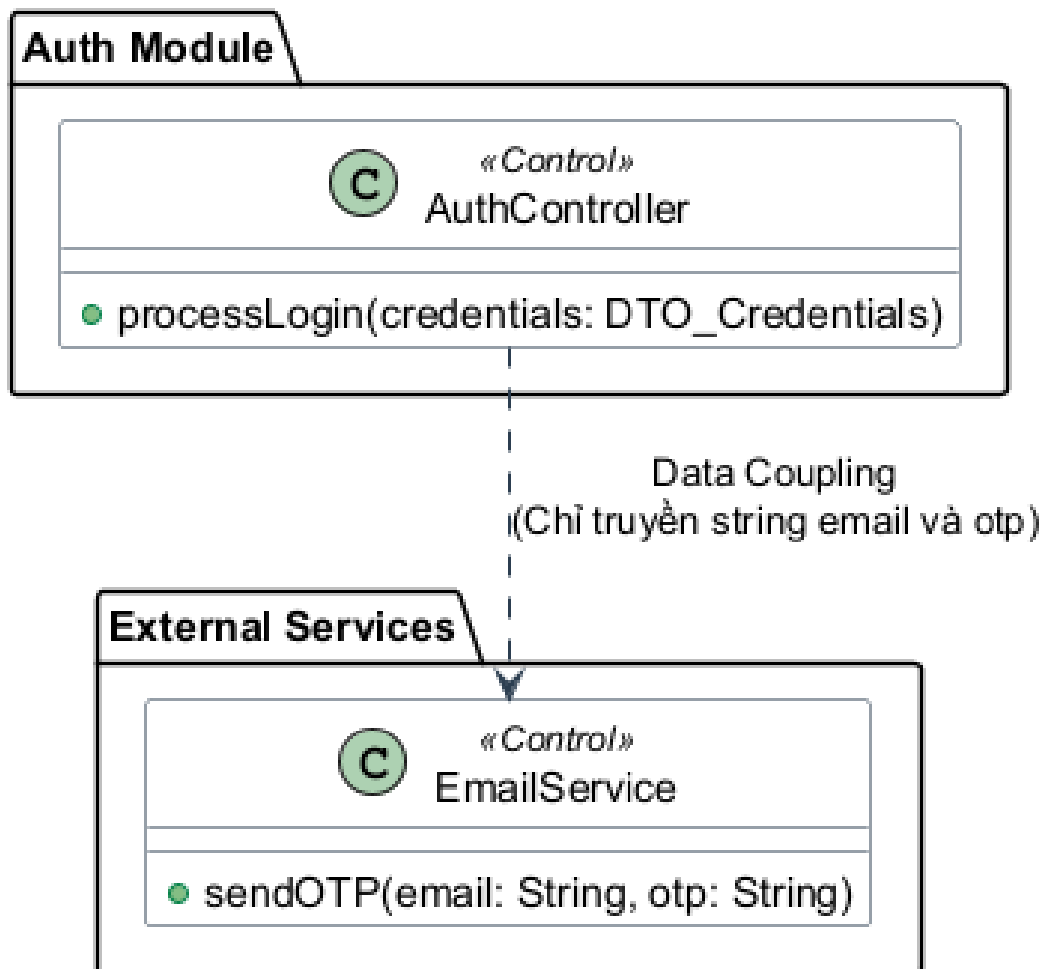
Đánh giá: Data Coupling (Tốt nhất)

Giải thích:

- Module Auth tương tác với các hệ thống bên ngoài (ví dụ gửi Email OTP) thông qua các tham số nguyên thủy (primitive data) hoặc đối tượng truyền tải (DTO).
- Không chia sẻ biến dùng chung toàn cục (Global Variable) nào với các module khác, đảm bảo khi sửa module Auth không ảnh hưởng đến Family hay Pantry.

Functional Cohesion - Module 1 (Auth)*Module 1 - Auth: Functional Cohesion Diagram*

Data Coupling - Module 1 (Auth)



Module 1 - Auth: Data Coupling Diagram

Đánh giá Coupling & Cohesion - Module 2 (Family)

1. Mức độ Cohesion (Gắn kết)

Đánh giá: Functional Cohesion

Giải thích:

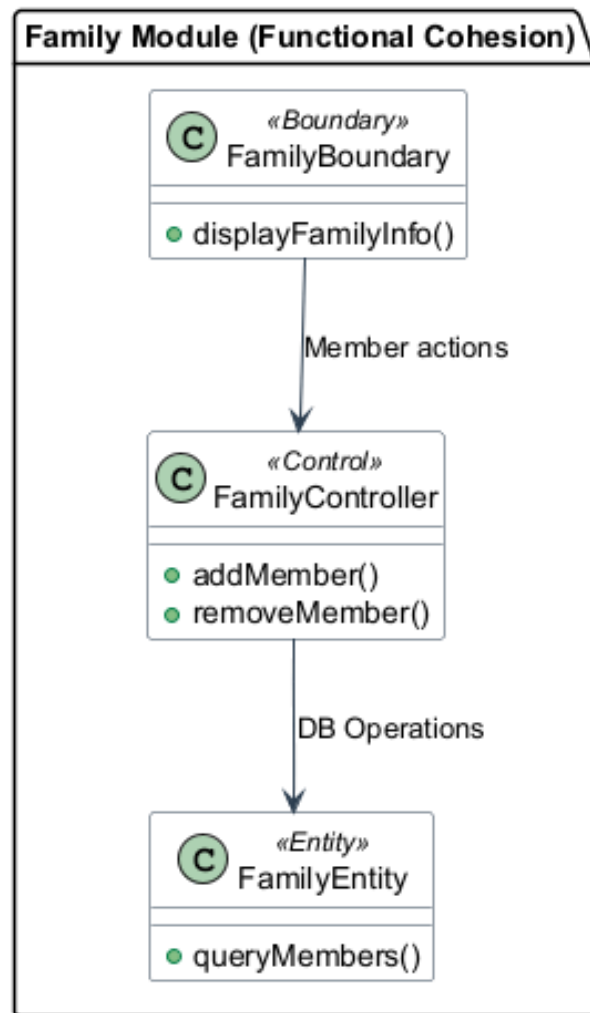
- Mọi chức năng liên quan đến việc tạo nhóm, rời nhóm, mời thành viên đều nằm trọn trong module Family.
- `FamilyController` không chứa bất kỳ logic nào về việc quản lý thực phẩm của nhóm (việc đó để `Pantry` lo), do đó mức độ chuyên biệt và tập trung vào chức năng là tuyệt đối.

2. Mức độ Coupling (Phụ thuộc)

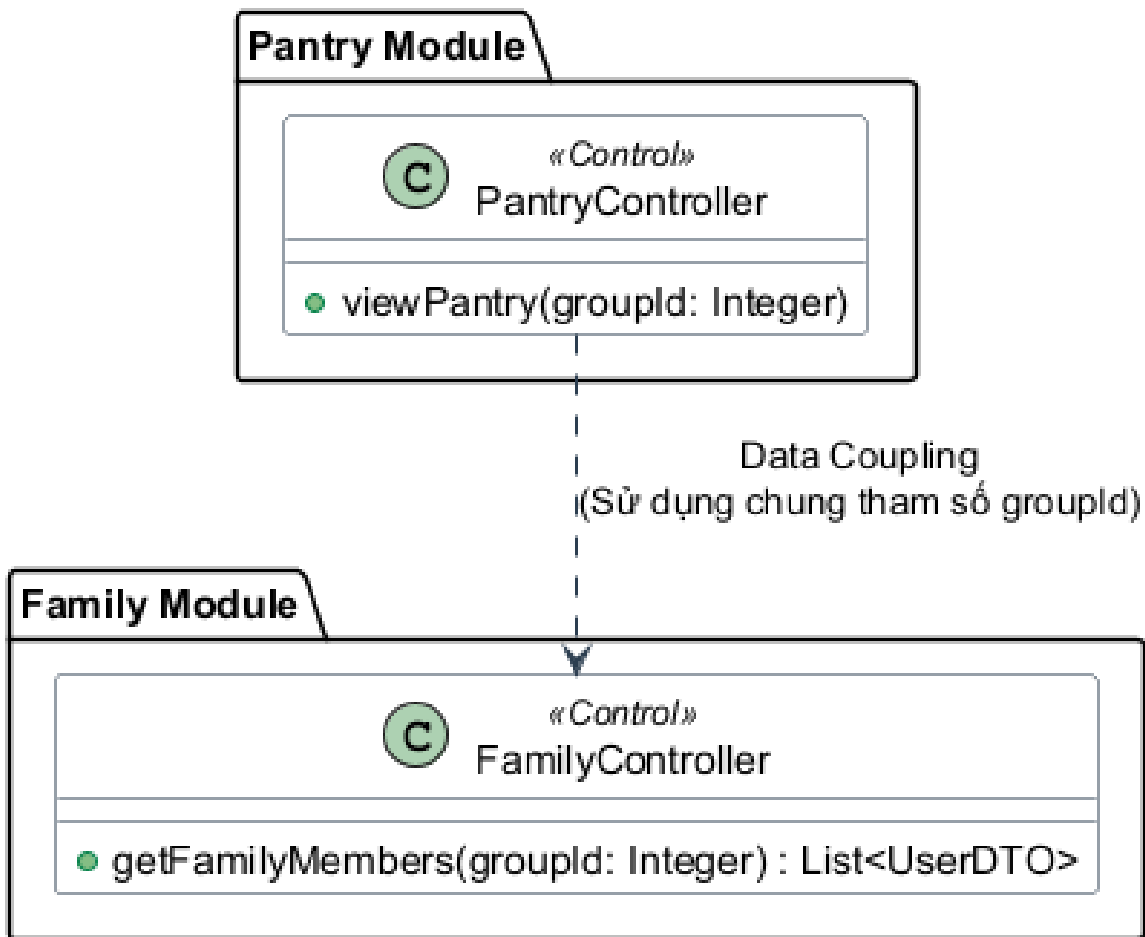
Đánh giá: Data Coupling

Giải thích:

- Module Family cung cấp dữ liệu cho Module Pantry và Shopping thông qua việc trả về `groupId` dưới dạng tham số cơ bản (Primitive Data Type).
- Không có sự phụ thuộc lẫn lộn (Common Coupling), ví dụ: Pantry không gọi thẳng vào biến private của Family.

Functional Cohesion - Module 2 (Family)*Module 2 - Family: Functional Cohesion Diagram*

Data Coupling - Module 2 (Family)



Module 2 - Family: Data Coupling Diagram

Đánh giá Coupling & Cohesion - Module 3 (Shopping Lists)

1. Mức độ Cohesion (Gắn kết)

Đánh giá: Functional Cohesion

Giải thích:

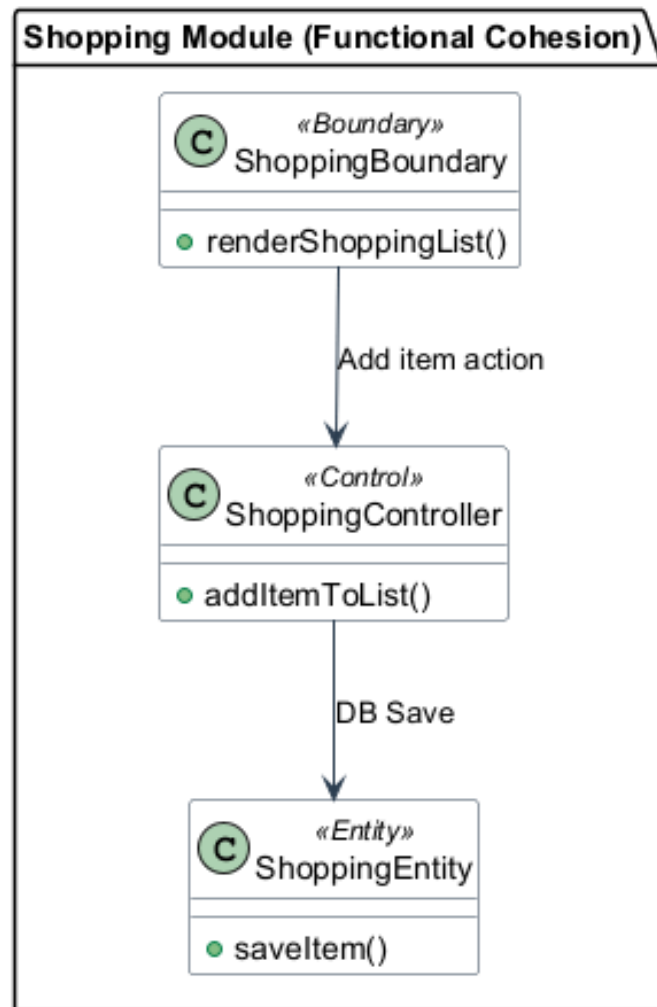
- Module tập trung giải quyết trọn vẹn luồng "Mua sắm": Tạo danh sách, thêm món đồ, đánh dấu đã mua.
- Không có bất kỳ logic nào lạc lõng lọt vào module này (Ví dụ: logic thêm món ăn vào tủ lạnh sẽ được gọi sang module Pantry thay vì tự viết code trực tiếp ở đây).

2. Mức độ Coupling (Phụ thuộc)

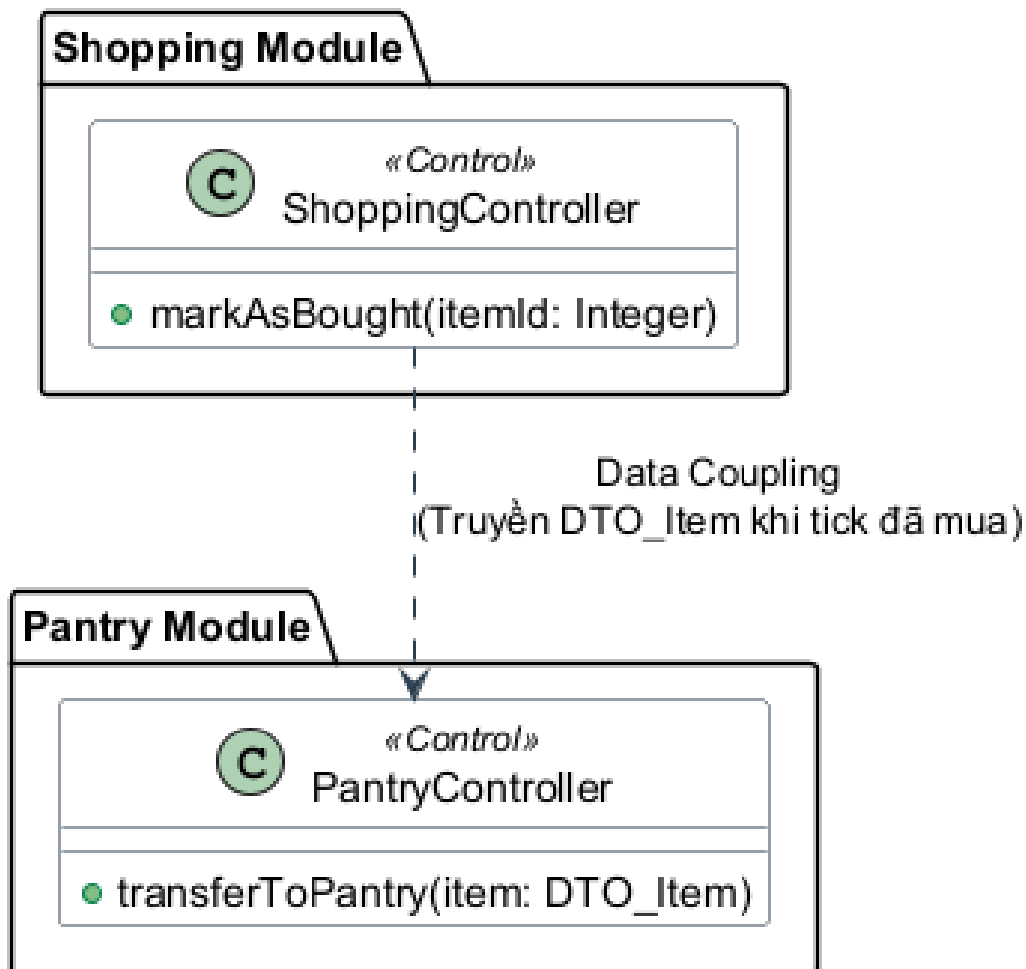
Đánh giá: Data Coupling

Giải thích:

- Khi người dùng đánh dấu mua một món (tick "Đã mua"), *ShoppingController* truyền *DTO_Item* sang *PantryController* để tủ lạnh tự động cộng số lượng.
- Nó truyền tham số rạch ròi, hoàn toàn không thao túng logic cập nhật Database nội bộ của Tủ lạnh (Content Coupling).

Functional Cohesion - Module 3 (Shopping Lists)*Module 3 - Shopping Lists: Functional Cohesion Diagram*

Data Coupling - Module 3 (Shopping Lists)



Module 3 - Shopping Lists: Data Coupling Diagram

Đánh giá Coupling & Cohesion - Module 4 (Pantry)

1. Mức độ Cohesion (Gắn kết)

Đánh giá: Functional Cohesion

Giải thích:

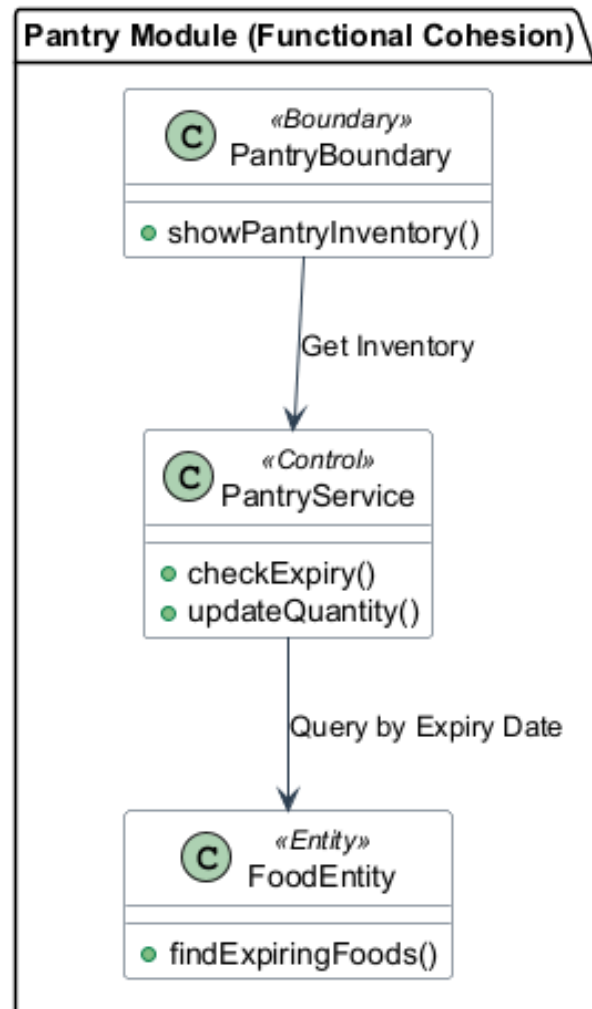
- Module Pantry tập hợp tất cả các method liên quan đến vòng đời thực phẩm: Thêm vào tủ, cập nhật số lượng, trừ số lượng, quản lý hạn sử dụng.
- Tất cả các thao tác đều xoay quanh một mục đích cốt lõi là Quản lý Kho lưu trữ (Pantry).

2. Mức độ Coupling (Phụ thuộc)

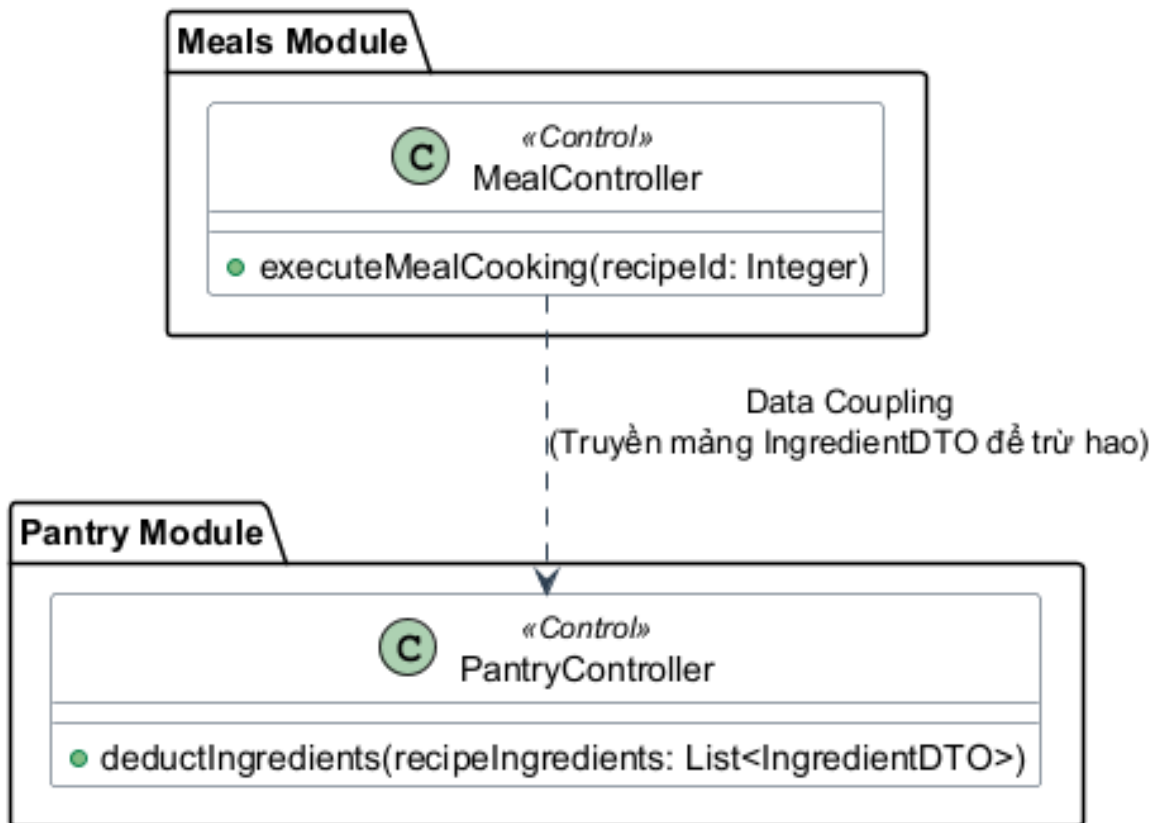
Đánh giá: Data Coupling

Giải thích:

- **Pantry** là trạm trung chuyển dữ liệu được nhiều module khác (như Meals, Shopping) gọi tới.
- Nó chỉ nhận/trả tham số `List<Ingredient>` hoặc `FoodID` thông qua các API công khai chứ không chia sẻ chung bộ nhớ hay dữ liệu Database thô, tuân thủ đúng nguyên lý Information Hiding.

Functional Cohesion - Module 4 (Pantry)*Module 4 - Pantry: Functional Cohesion Diagram*

Data Coupling - Module 4 (Pantry)



Module 4 - Pantry: Data Coupling Diagram

Đánh giá Coupling & Cohesion - Module 5 (Meals)

1. Mức độ Cohesion (Gắn kết)

Đánh giá: Functional Cohesion

Giải thích:

- Module Meals đảm nhiệm chức năng tạo Lịch trình ăn (Meal Plan) và quản lý các phiên nấu ăn (Cooking Session).
- Mọi logic bên trong module (chọn ngày, chọn món, tính toán calo của bữa ăn) đều hướng đến một mục đích chung duy nhất là xây dựng bữa ăn hoàn chỉnh cho gia đình.

2. Mức độ Coupling (Phụ thuộc)

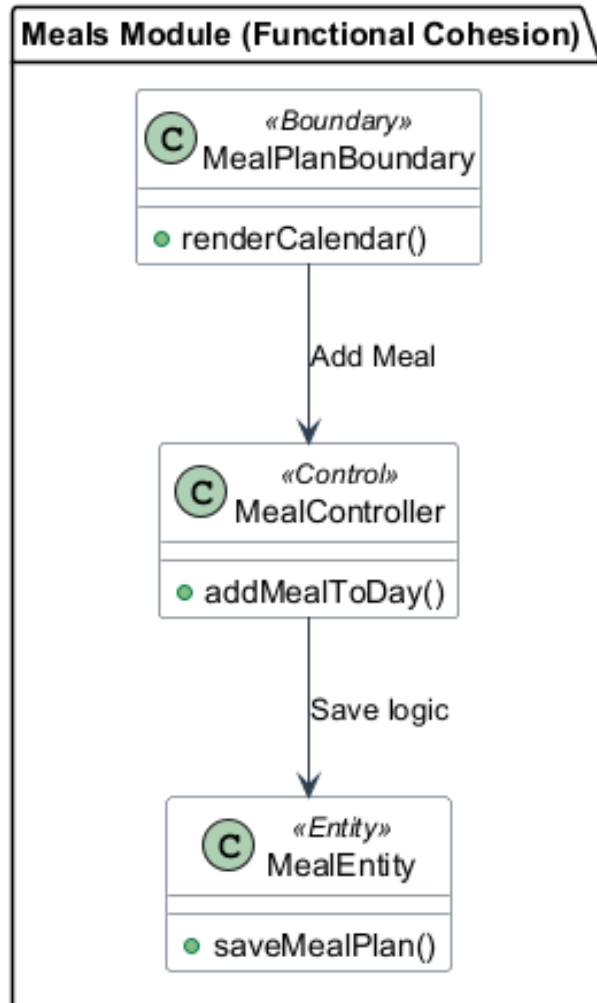
Đánh giá: Data Coupling

Giải thích:

- Khi Meals cần nguyên liệu để nấu, nó không tự tiện chọc vào Database của Pantry để trừ. Thay vào đó nó gọi hàm `deductIngredients(list)` của `PantryController`.

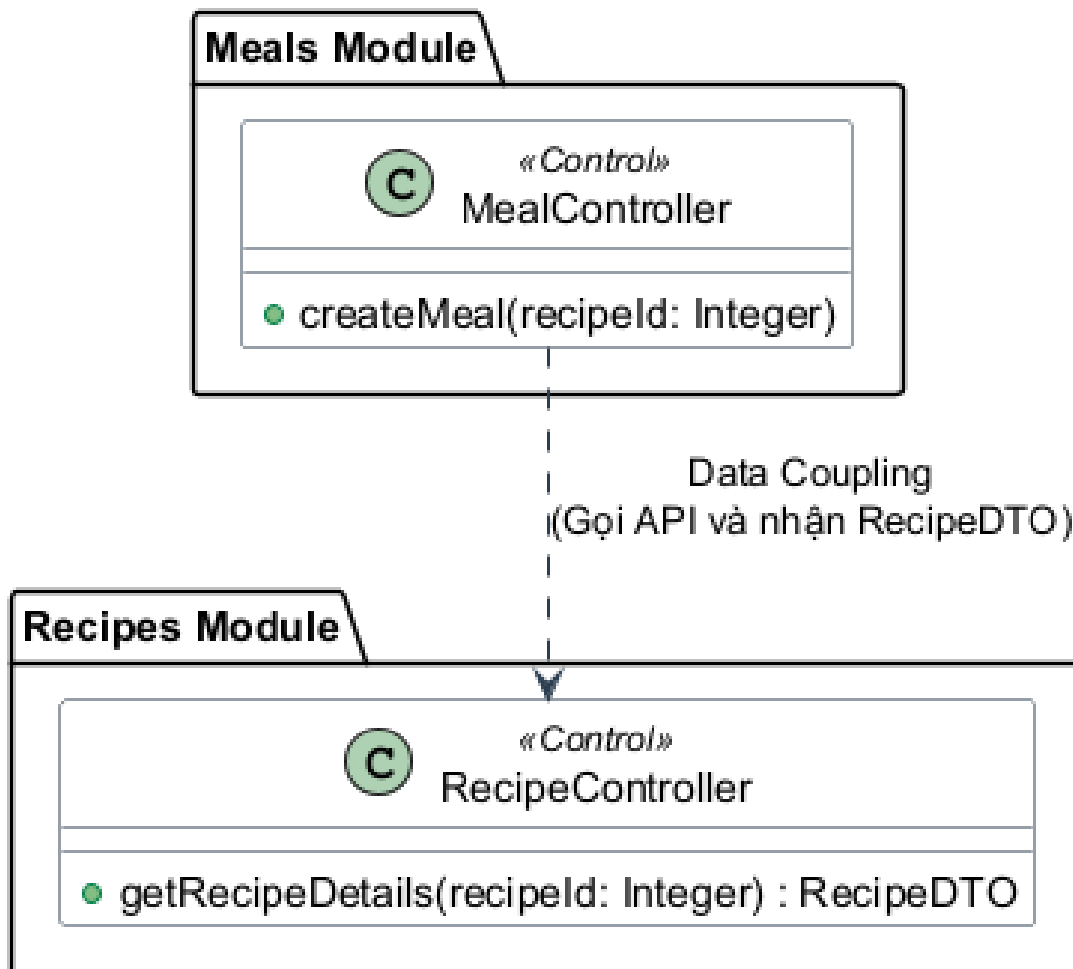
- Khi cần công thức nấu ăn, nó nhận một đối tượng `RecipeDTO` từ module Recipes thay vì tự quản lý dữ liệu công thức.

Functional Cohesion - Module 5 (Meals)



Module 5 - Meals: Functional Cohesion Diagram

Data Coupling - Module 5 (Meals)



Module 5 - Meals: Data Coupling Diagram

Đánh giá Coupling & Cohesion - Module 6 (Recipes & AI Chef)

1. Mức độ Cohesion (Gắn kết)

Đánh giá: Functional Cohesion

Giải thích:

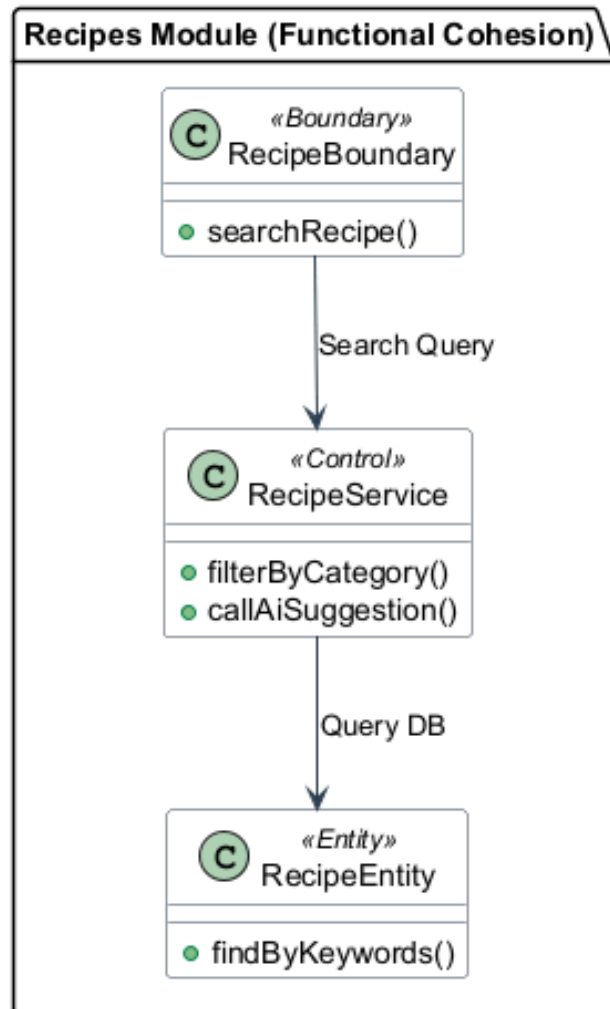
- Module tập trung cao độ vào việc tìm kiếm, gợi ý và xử lý công thức nấu ăn.
- Ngay cả AI Chef (một tính năng phức tạp) cũng được gom vào đây vì bản chất nó phục vụ cho việc tạo hoặc giải đáp các thắc mắc xoay quanh "Công thức" (Recipe). Điều này giúp code gọn gàng, tránh bị phân tán (Coincidental Cohesion).

2. Mức độ Coupling (Phụ thuộc)

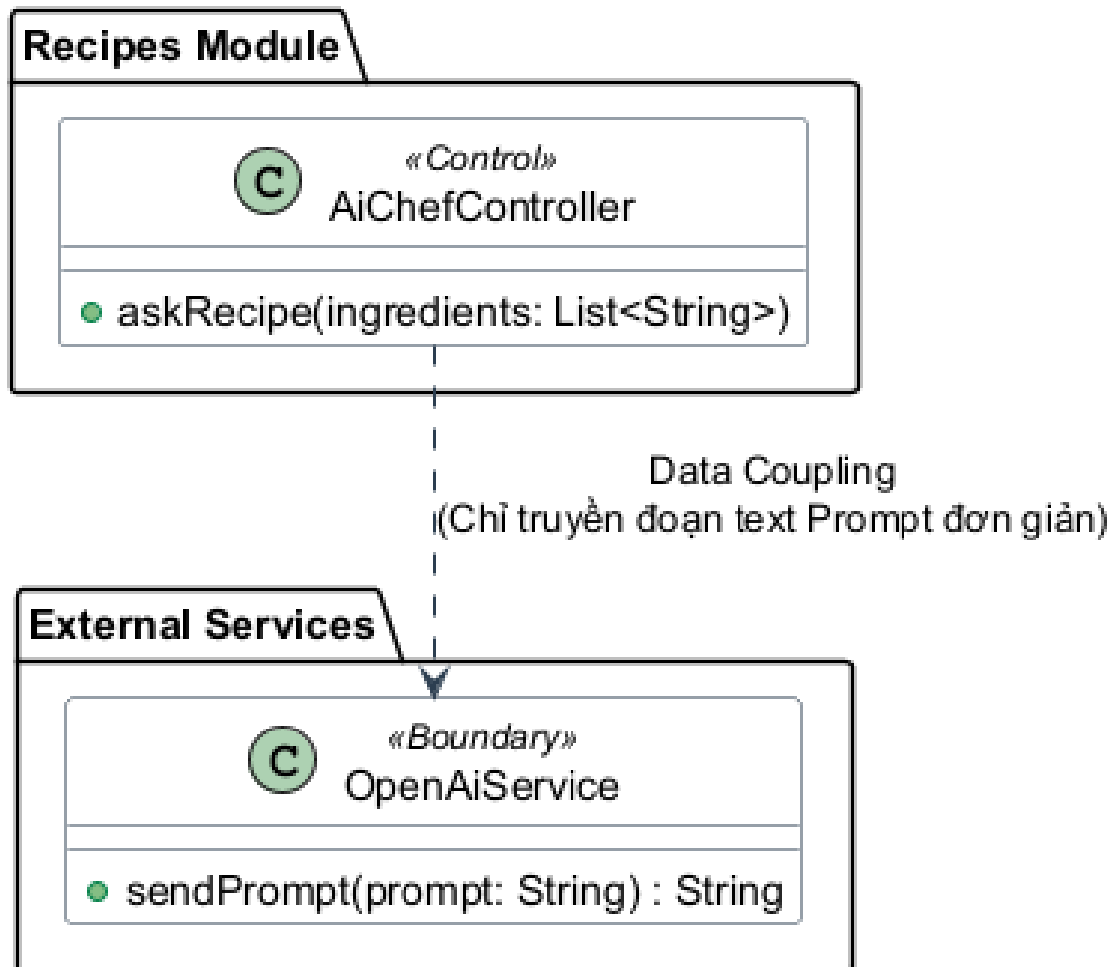
Đánh giá: Data Coupling

Giải thích:

- Khi module AI Chef tương tác với External API của ChatGPT, nó không truyền toàn bộ Database sang, mà chỉ truyền những tham số cần thiết (List of ingredients) bằng định dạng chuỗi JSON. Điều này tạo ra sự lỏng lẻo tuyệt đối giữa App và nền tảng AI bên thứ ba.

Functional Cohesion - Module 6 (Recipes & AI Chef)*Module 6 - Recipes: Functional Cohesion Diagram*

Data Coupling - Module 6 (Recipes & AI Chef)



Module 6 - Recipes: Data Coupling Diagram

Đánh giá Coupling & Cohesion - Module 7 (Reports)

1. Mức độ Cohesion (Gắn kết)

Đánh giá: Functional Cohesion

Giải thích:

- Module tập hợp tất cả các hàm logic cần thiết để tạo ra các báo cáo tổng quan. Việc gom nhóm như thế này tuân thủ nguyên tắc functional vì tất cả cùng phục vụ một mục tiêu hiển thị Dashboard và Biểu đồ cho người dùng.

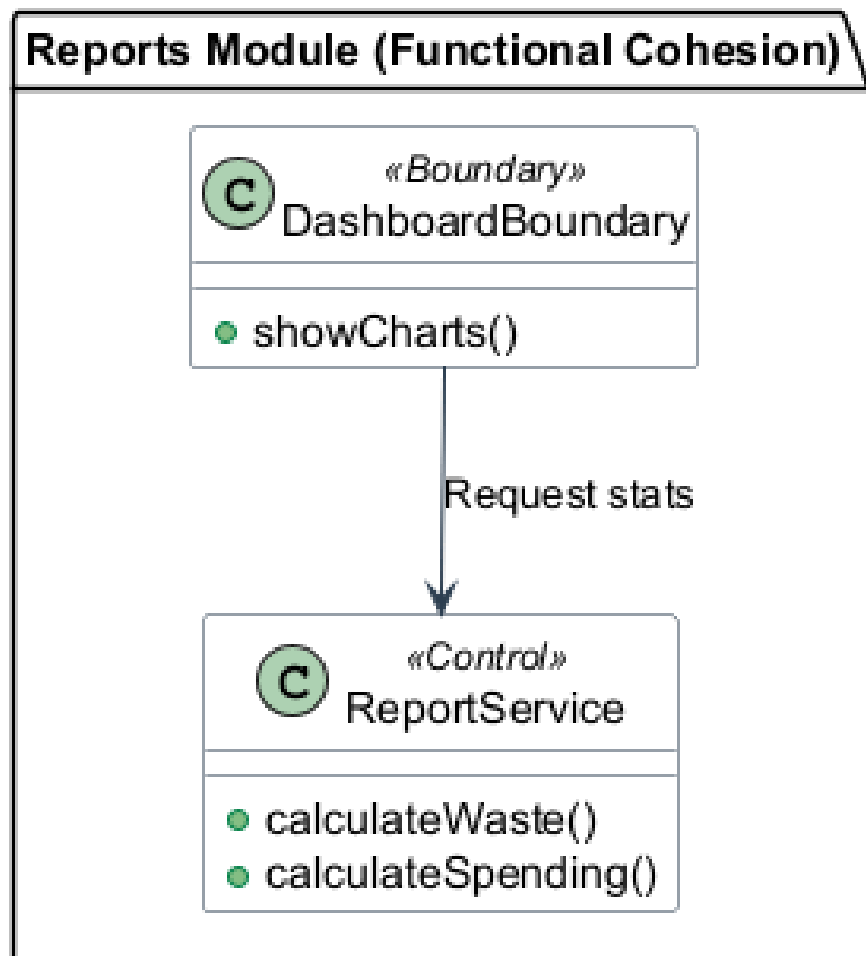
2. Mức độ Coupling (Phụ thuộc)

Đánh giá: Data Coupling

Giải thích:

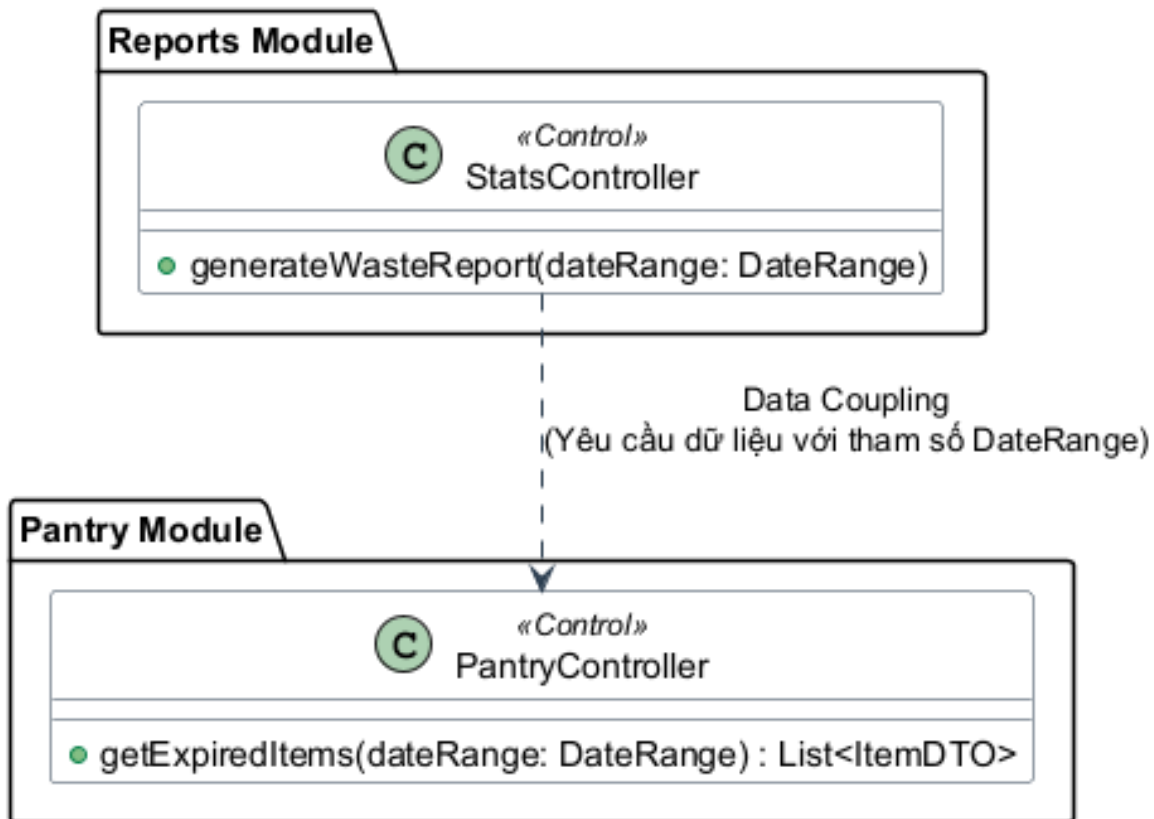
- Module Reports chỉ đóng vai trò "Read-only" đối với các module khác. Nó truyền các Data Transfer Objects hoặc ID (ví dụ `userId`, `dateRange`) tới Controller của Pantry hoặc Shopping để lấy số liệu về. Không can thiệp chỉnh sửa dữ liệu gốc.

Functional Cohesion - Module 7 (Reports)



Module 7 - Reports: Functional Cohesion Diagram

Data Coupling - Module 7 (Reports)



Module 7 - Reports: Data Coupling Diagram

Đánh giá Coupling & Cohesion - Module 8 (Admin)

1. Mức độ Cohesion (Gắn kết)

Đánh giá: Functional Cohesion

Giải thích:

- Module Admin thực hiện tất cả các tác vụ quản trị hệ thống (Duyệt công thức, Khóa tài khoản, Xem tổng quan). Nó cô lập mọi nghiệp vụ rủi ro cao vào chung một module.

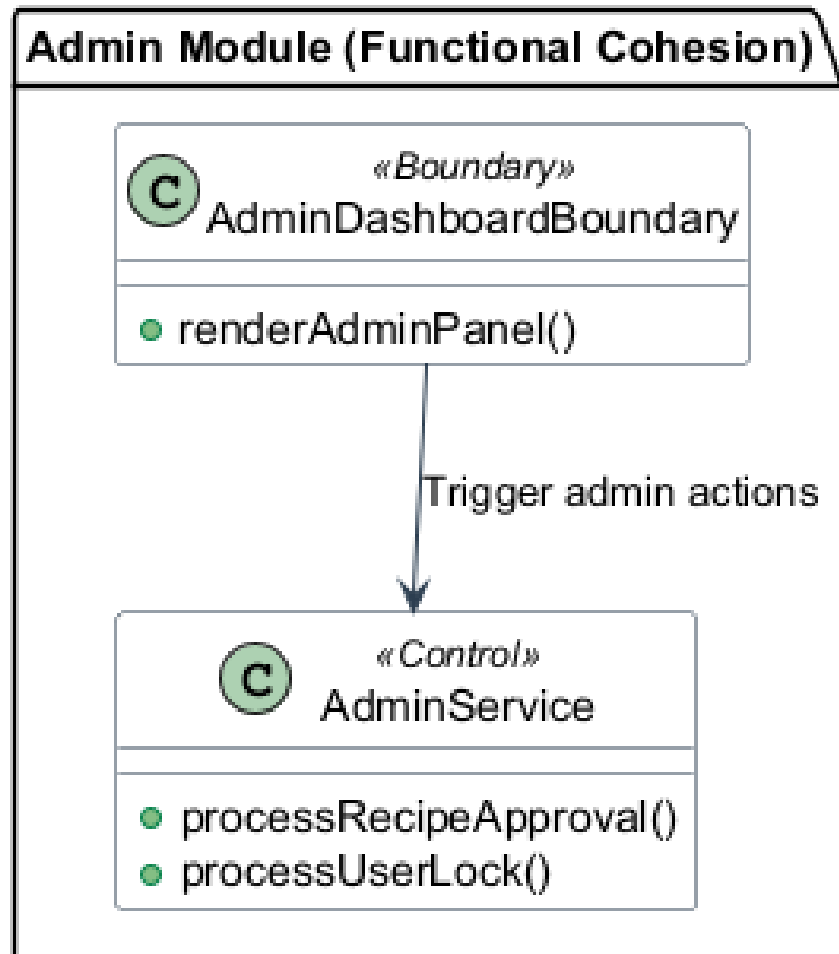
2. Mức độ Coupling (Phụ thuộc)

Đánh giá: Data Coupling

Giải thích:

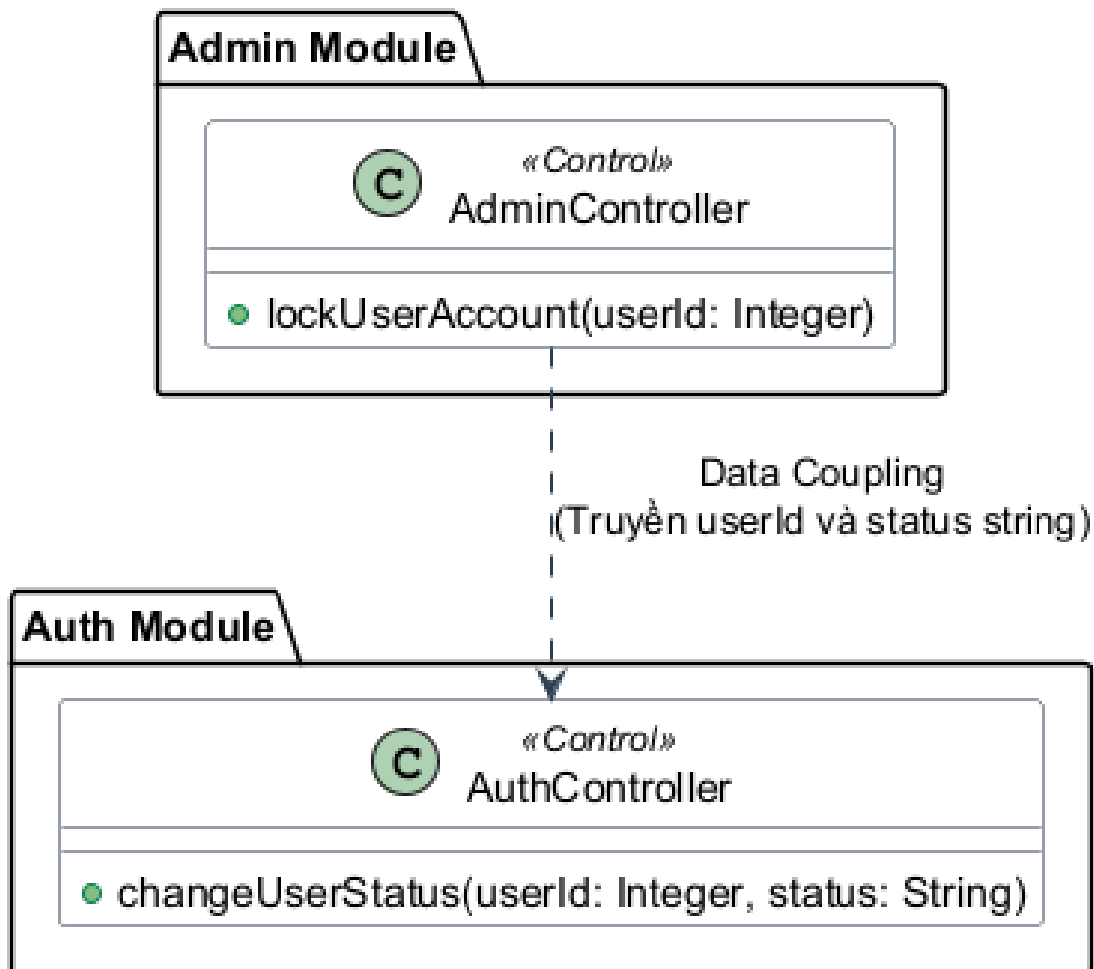
- Admin gọi sang module Auth để đổi trạng thái khóa tài khoản của User bằng cách gọi API nội bộ và truyền **UserId**. Nó không dùng chung database connection hay thao tác chọc thẳng vào memory của module Auth.

Functional Cohesion - Module 8 (Admin)



Module 8 - Admin: Functional Cohesion Diagram

Data Coupling - Module 8 (Admin)



Module 8 - Admin: Data Coupling Diagram

Đánh giá Coupling & Cohesion - Module 9 (Shared)

1. Mức độ Cohesion (Gắn kết)

Đánh giá: Functional Cohesion

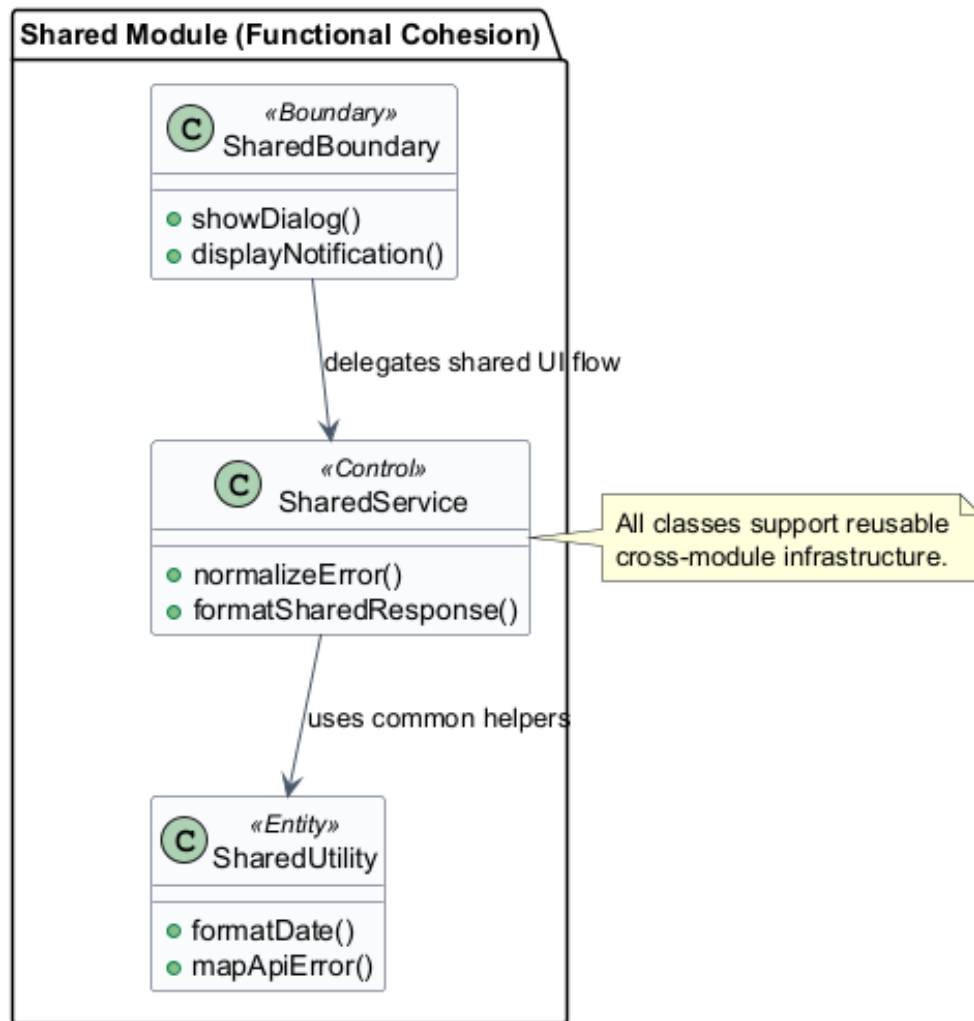
- Module Shared tập trung các thành phần dùng chung cho toàn hệ thống như context, API client, utility service và các helper không thuộc riêng một nghiệp vụ cụ thể.
- Các lớp trong module này cùng phục vụ mục tiêu giảm lặp lại và chuẩn hóa cách các module khác tương tác với hạ tầng chung.
- Việc gom các thành phần dùng chung vào Shared giúp tránh trộn logic hỗ trợ vào Auth, Pantry, Meals hoặc Admin, qua đó giữ cohesion của từng module nghiệp vụ.

2. Mức độ Coupling (Phụ thuộc)

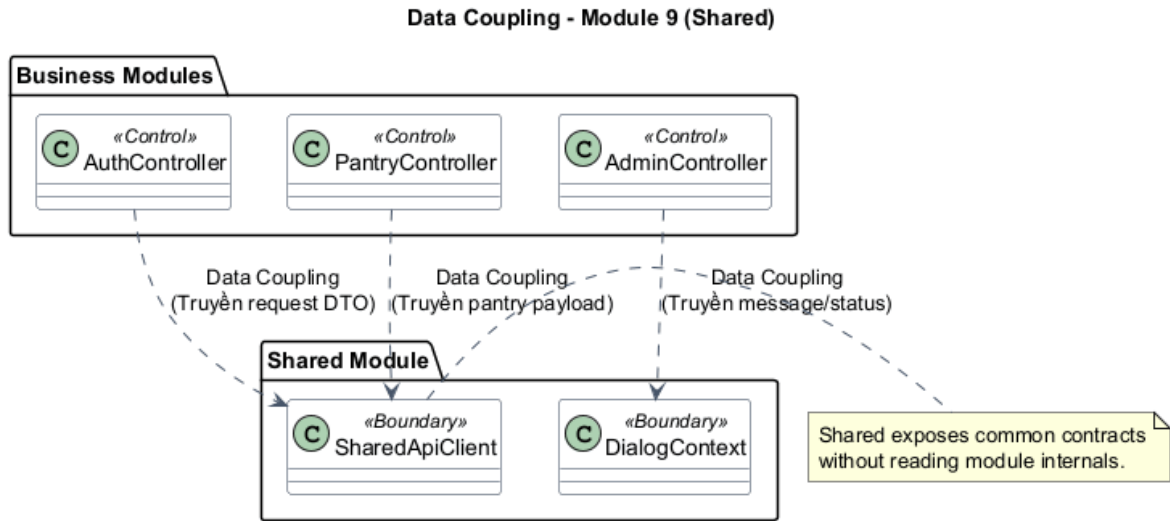
Đánh giá: Data Coupling

- Các module khác giao tiếp với Shared thông qua tham số, DTO hoặc interface rõ ràng, không truy cập trực tiếp trạng thái nội bộ của Shared.
- Shared cung cấp dịch vụ nền tảng nhưng không phụ thuộc ngược vào nghiệp vụ cụ thể của từng module, nhờ đó tránh Common Coupling và Content Coupling.
- Khi cần thay đổi cơ chế dialog, API client hoặc service dùng chung, các module nghiệp vụ chỉ cần giữ nguyên contract đang sử dụng.

Functional Cohesion - Module 9 (Shared)



Module 9 - Shared: Functional Cohesion Diagram



Phụ lục: Diagram đầy đủ

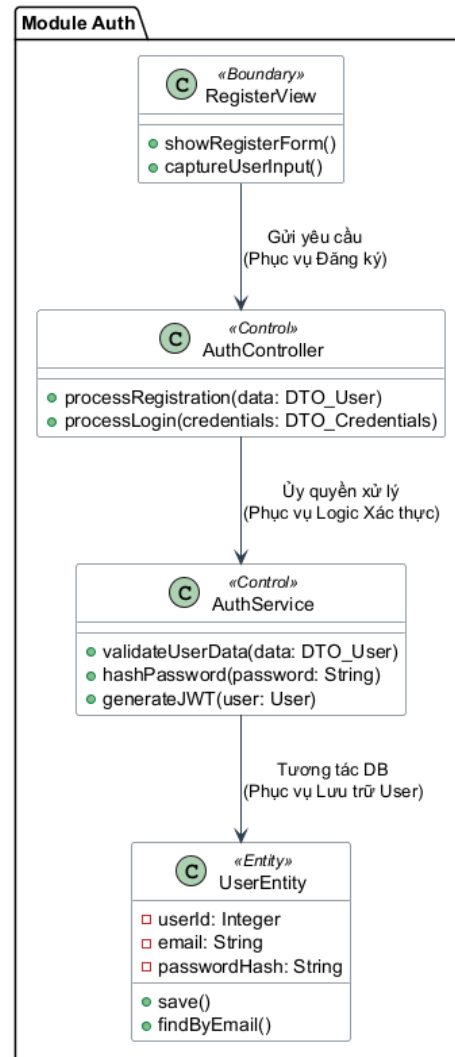
Phụ lục này liệt kê đầy đủ diagram trong phần GoodDesign. Các diagram đã được render thành ảnh PNG và chèn trực tiếp vào tài liệu.

No.	Diagram	Trạng thái trong Word
1	Cohesion_Diagram	Đã render PNG và chèn vào Word
2	Coupling_Diagram	Đã render PNG và chèn vào Word
3	Module_1_Auth_Cohesion_Diagram	Đã render PNG và chèn vào Word
4	Module_1_Auth_Coupling_Diagram	Đã render PNG và chèn vào Word
5	Module_2_Family_Cohesion_Diagram	Đã render PNG và chèn vào Word
6	Module_2_Family_Coupling_Diagram	Đã render PNG và chèn vào Word
7	Module_3_Shopping_Lists_Cohesion_Diagram	Đã render PNG và chèn vào Word
8	Module_3_Shopping_Lists_Coupling_Diagram	Đã render PNG và chèn vào Word
9	Module_4_Pantry_Cohesion_Diagram	Đã render PNG và chèn vào Word
10	Module_4_Pantry_Coupling_Diagram	Đã render PNG và chèn vào Word
11	Module_5_Meals_Cohesion_Diagram	Đã render PNG và chèn vào Word
12	Module_5_Meals_Coupling_Diagram	Đã render PNG và chèn vào Word
13	Module_6_Recipes_Cohesion_Diagram	Đã render PNG và chèn vào Word
14	Module_6_Recipes_Coupling_Diagram	Đã render PNG và chèn vào Word
15	Module_7_Reports_Cohesion_Diagram	Đã render PNG và chèn vào Word
16	Module_7_Reports_Coupling_Diagram	Đã render PNG và chèn vào Word
17	Module_8_Admin_Cohesion_Diagram	Đã render PNG và chèn vào Word
18	Module_8_Admin_Coupling_Diagram	Đã render PNG và chèn vào Word
19	Module_9_Shared_Cohesion_Diagram	Đã render PNG và chèn vào Word
20	Module_9_Shared_Coupling_Diagram	Đã render PNG và chèn vào Word

Diagram A.1: Cohesion_Diagram

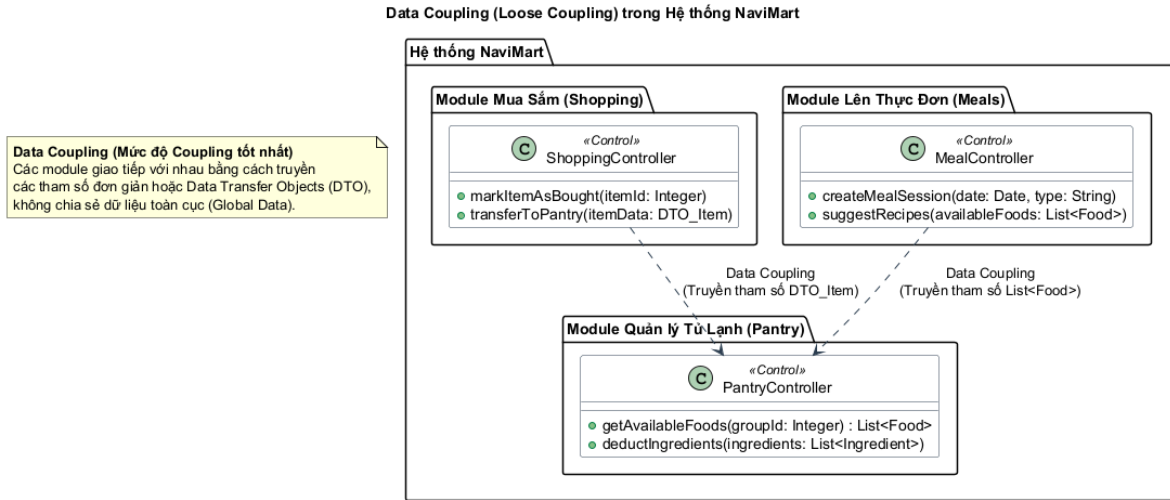
Functional Cohesion (Tight Cohesion) trong Module Auth

Functional Cohesion (Mức độ Cohesion tốt nhất)
Tất cả các thành phần trong module đều hướng tới và phục vụ cho một chức năng/trách nhiệm duy nhất: Quản lý Xác thực và Tài khoản Người dùng.



Cohesion_Diagram

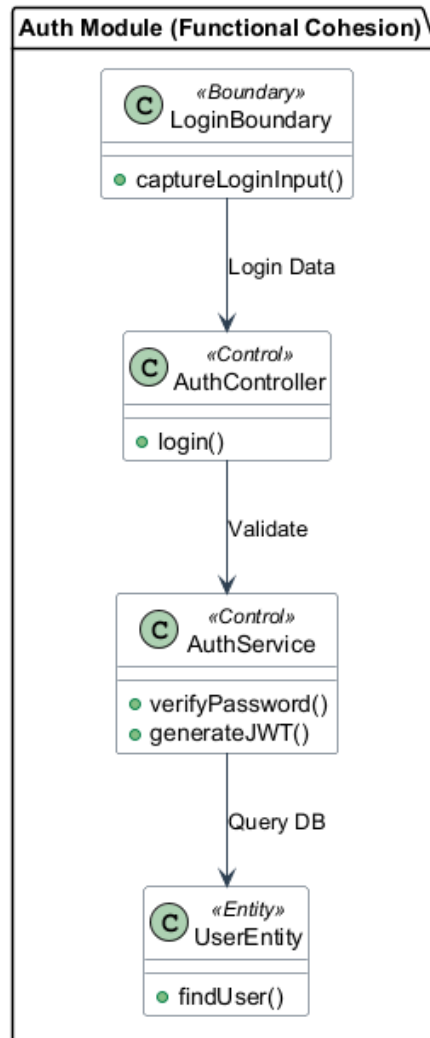
Diagram A.2: Coupling_Diagram



Coupling_Diagram

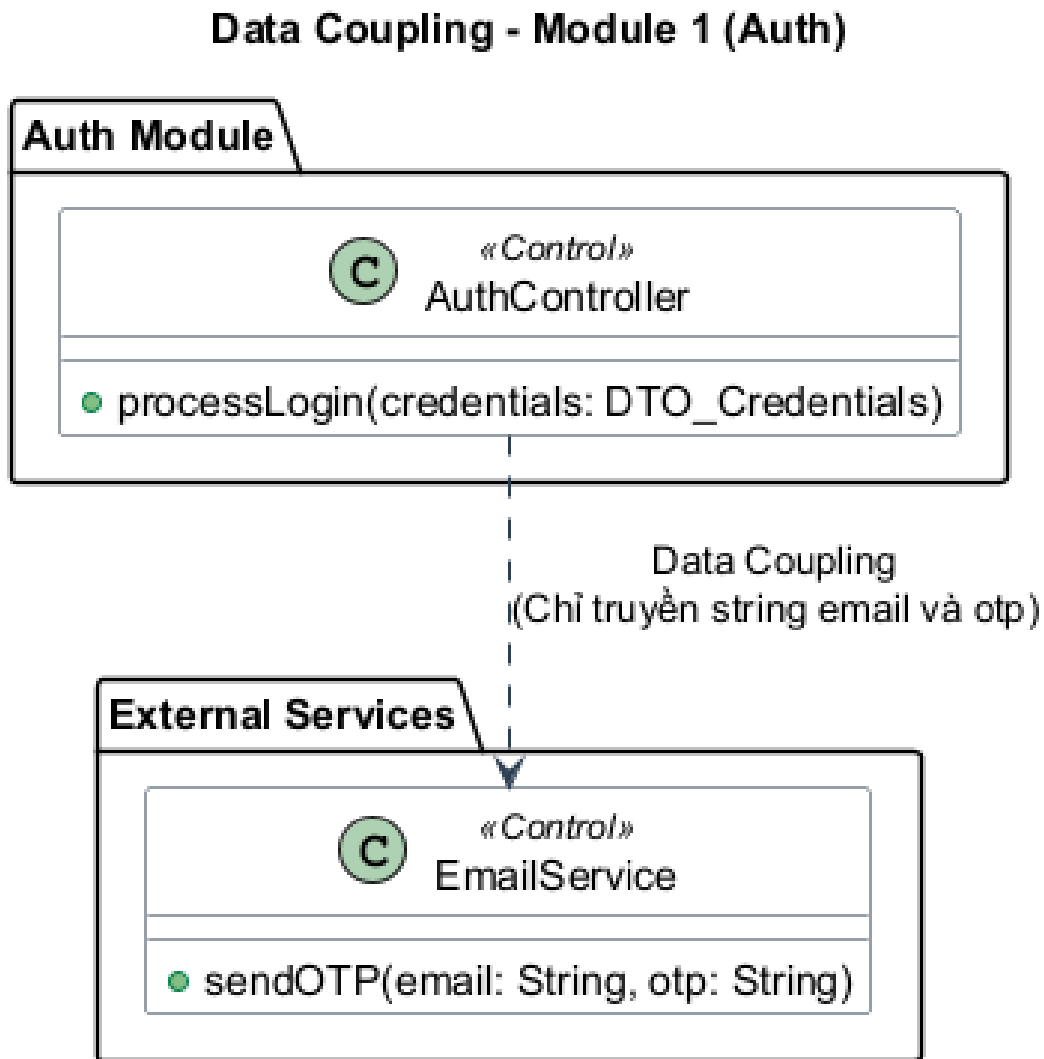
Diagram A.3: Module_1_Auth_Cohesion_Diagram

Functional Cohesion - Module 1 (Auth)



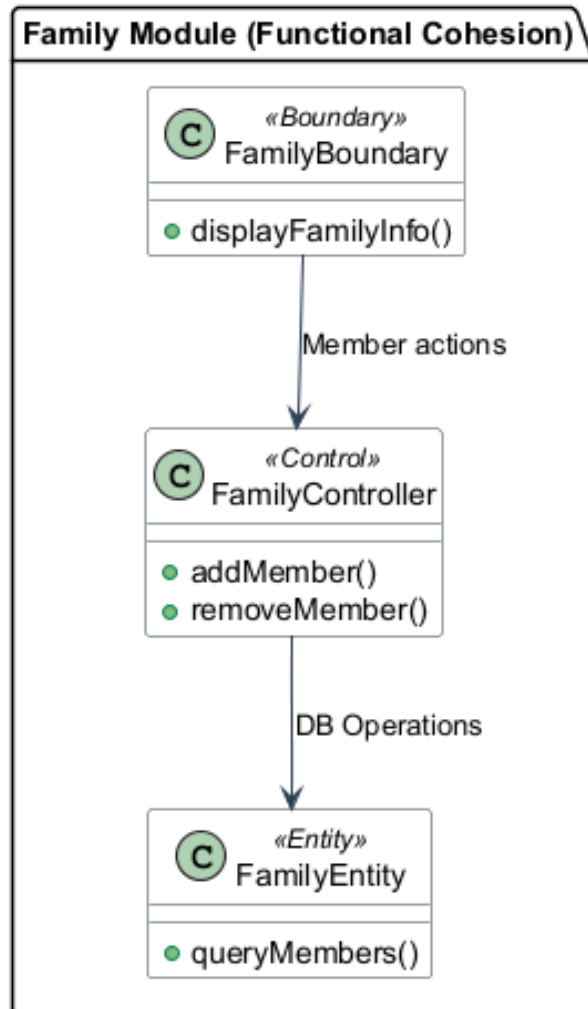
Module_1_Auth_Cohesion_Diagram

Diagram A.4: Module_1_Auth_Coupling_Diagram



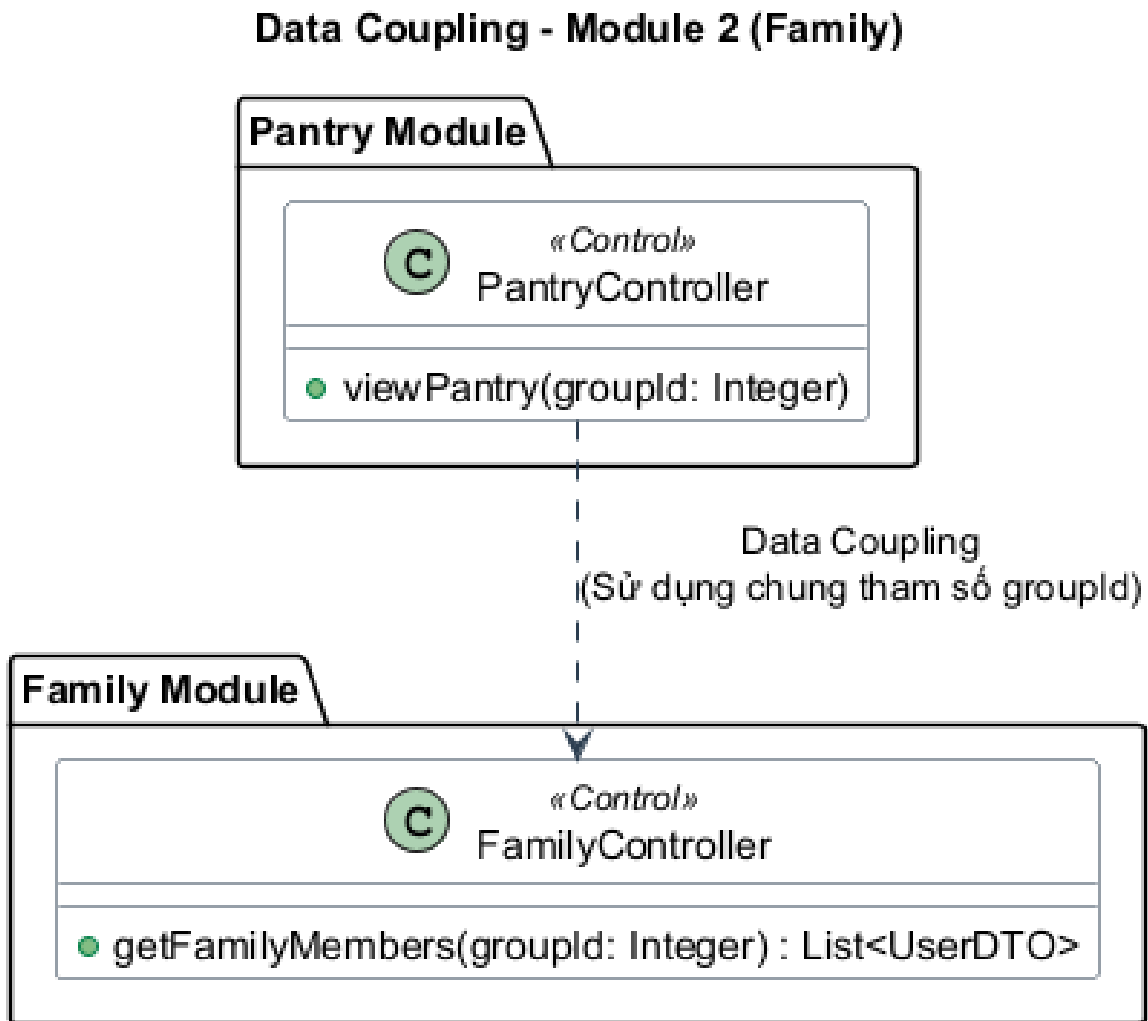
Module_1_Auth_Coupling_Diagram

Diagram A.5: Module_2_Family_Cohesion_Diagram

Functional Cohesion - Module 2 (Family)

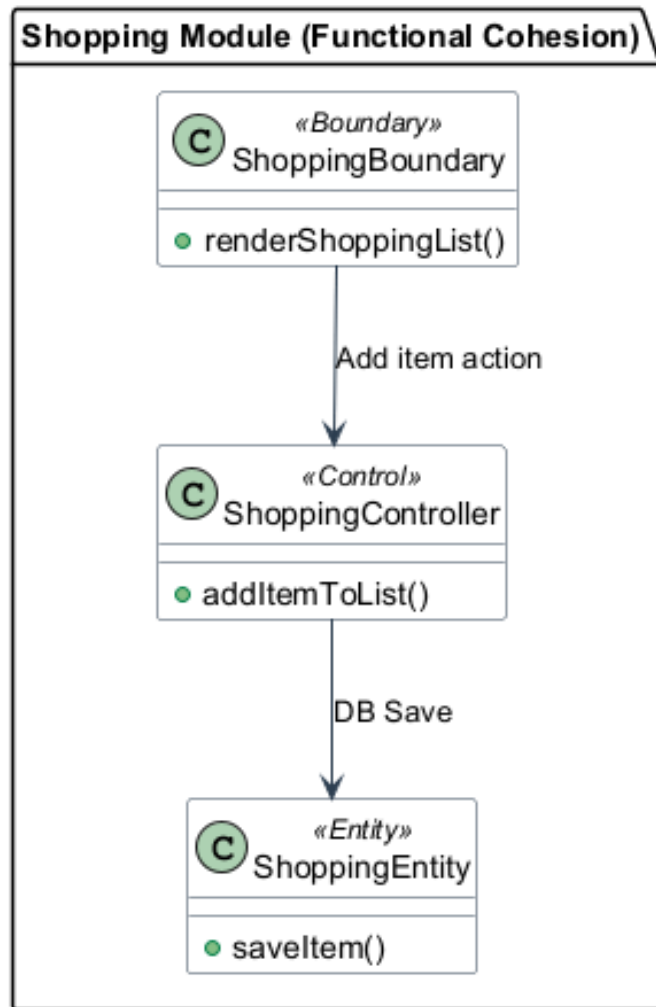
Module_2_Family_Cohesion_Diagram

Diagram A.6: Module_2_Family_Coupling_Diagram



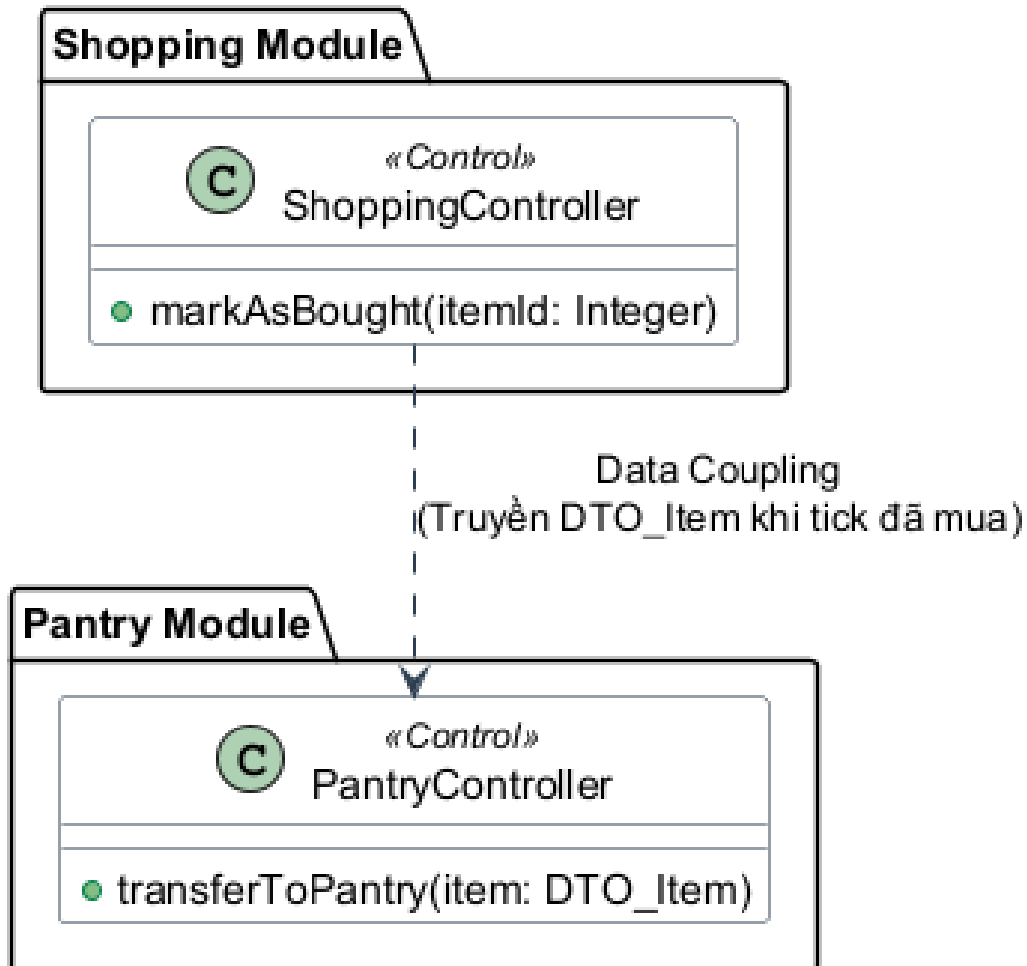
Module_2_Family_Coupling_Diagram

Diagram A.7: Module_3_Shopping_Lists_Cohesion_Diagram

Functional Cohesion - Module 3 (Shopping Lists)

Module_3_Shopping_Lists_Cohesion_Diagram

Diagram A.8: Module_3_Shopping_Lists_Coupling_Diagram

Data Coupling - Module 3 (Shopping Lists)

Module_3_Shopping_Lists_Coupling_Diagram

Diagram A.9: Module_4_Pantry_Cohesion_Diagram

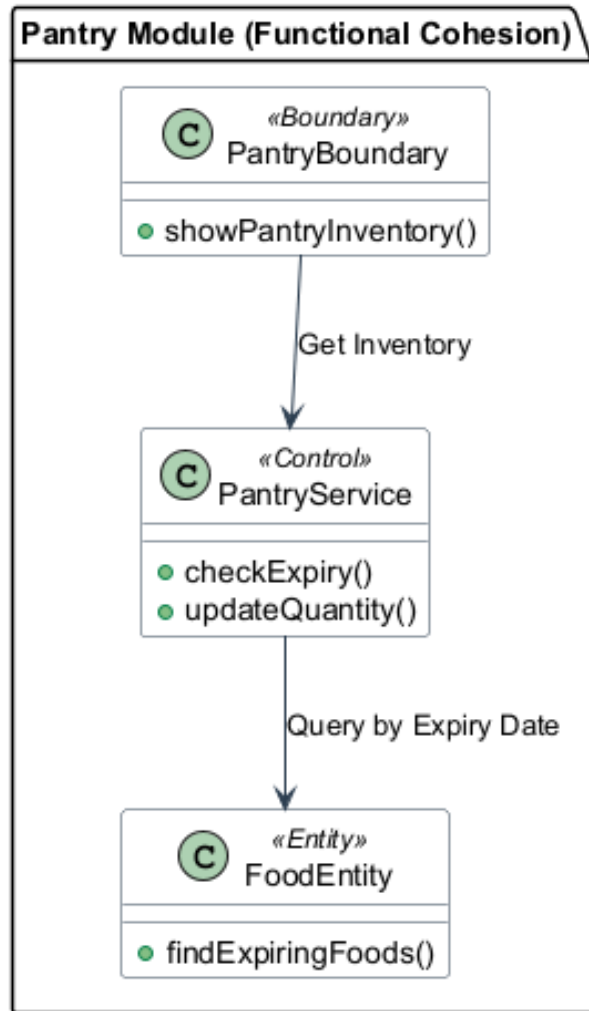
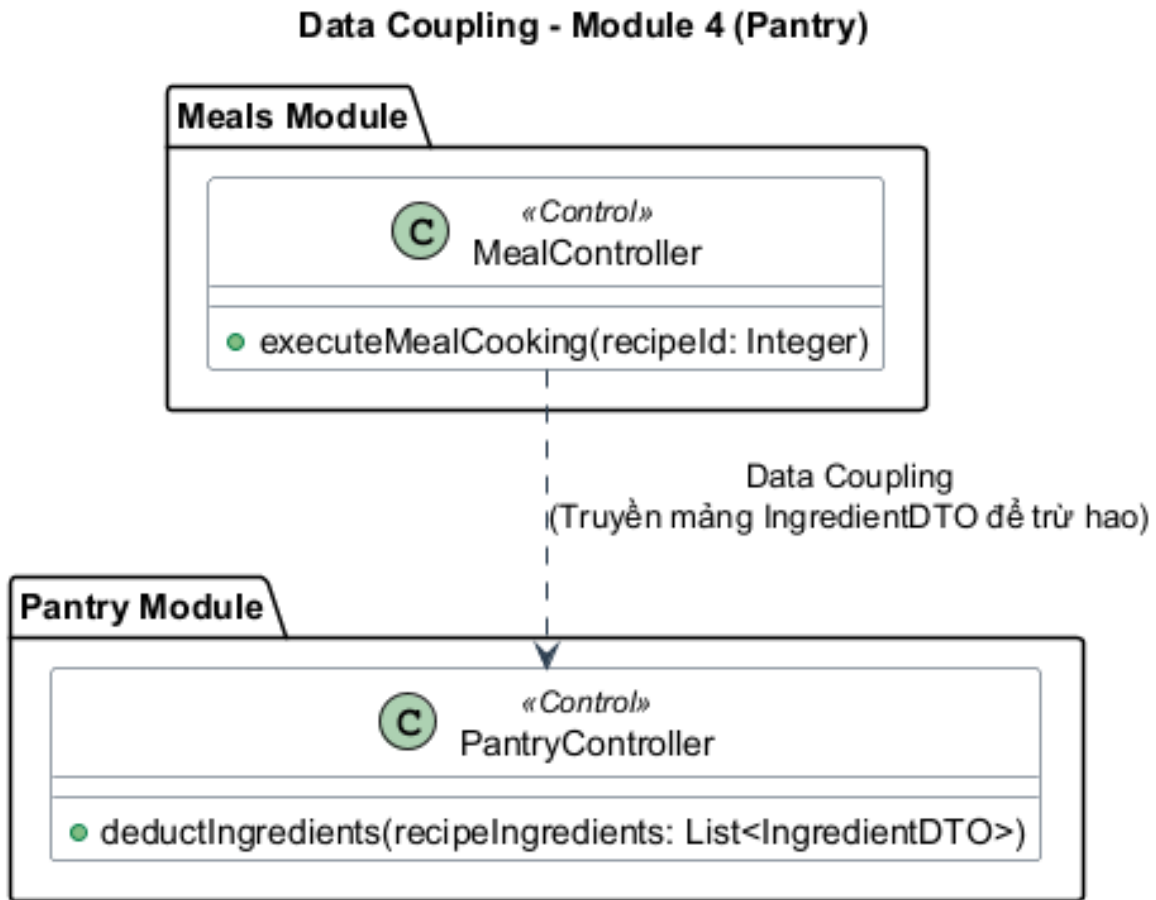
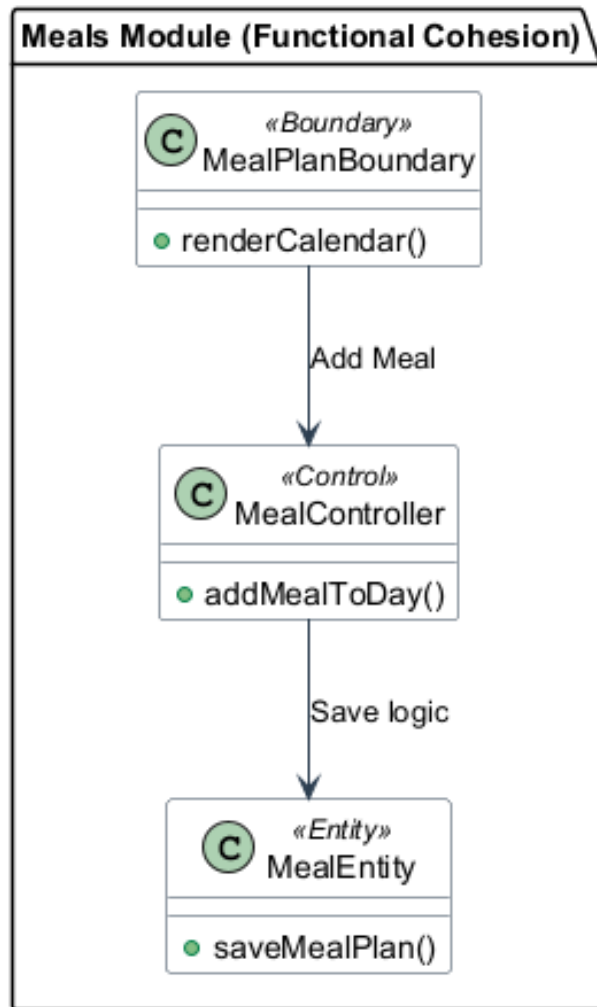
Functional Cohesion - Module 4 (Pantry)*Module_4_Pantry_Cohesion_Diagram*

Diagram A.10: Module_4_Pantry_Coupling_Diagram



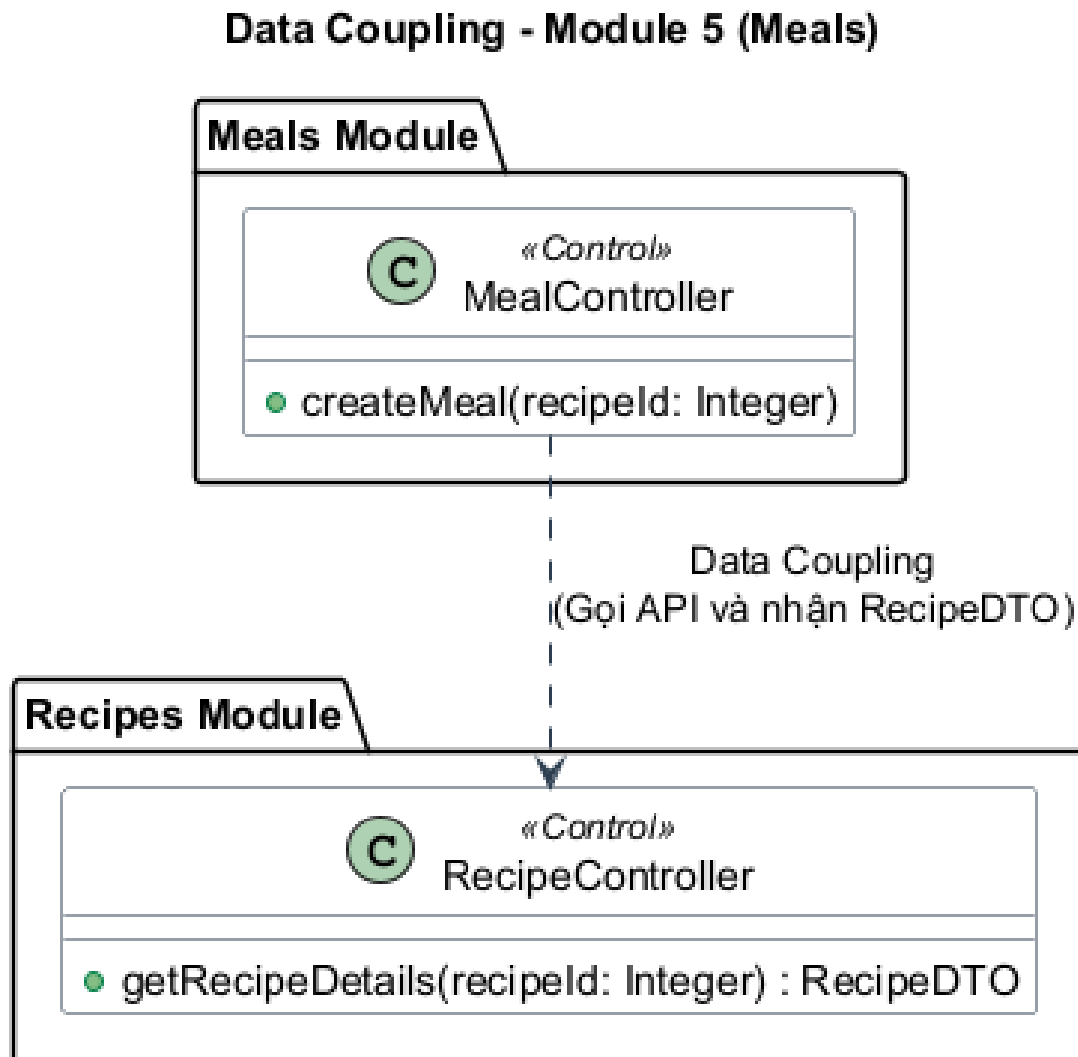
Module_4_Pantry_Coupling_Diagram

Diagram A.11: Module_5_Meals_Cohesion_Diagram

Functional Cohesion - Module 5 (Meals)

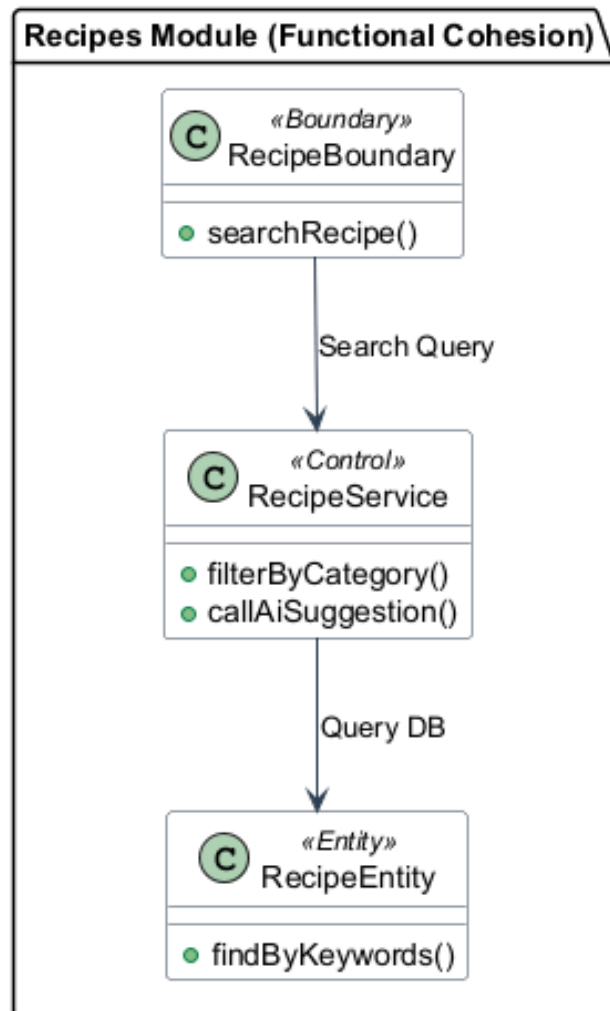
Module_5_Meals_Cohesion_Diagram

Diagram A.12: Module_5_Meals_Coupling_Diagram



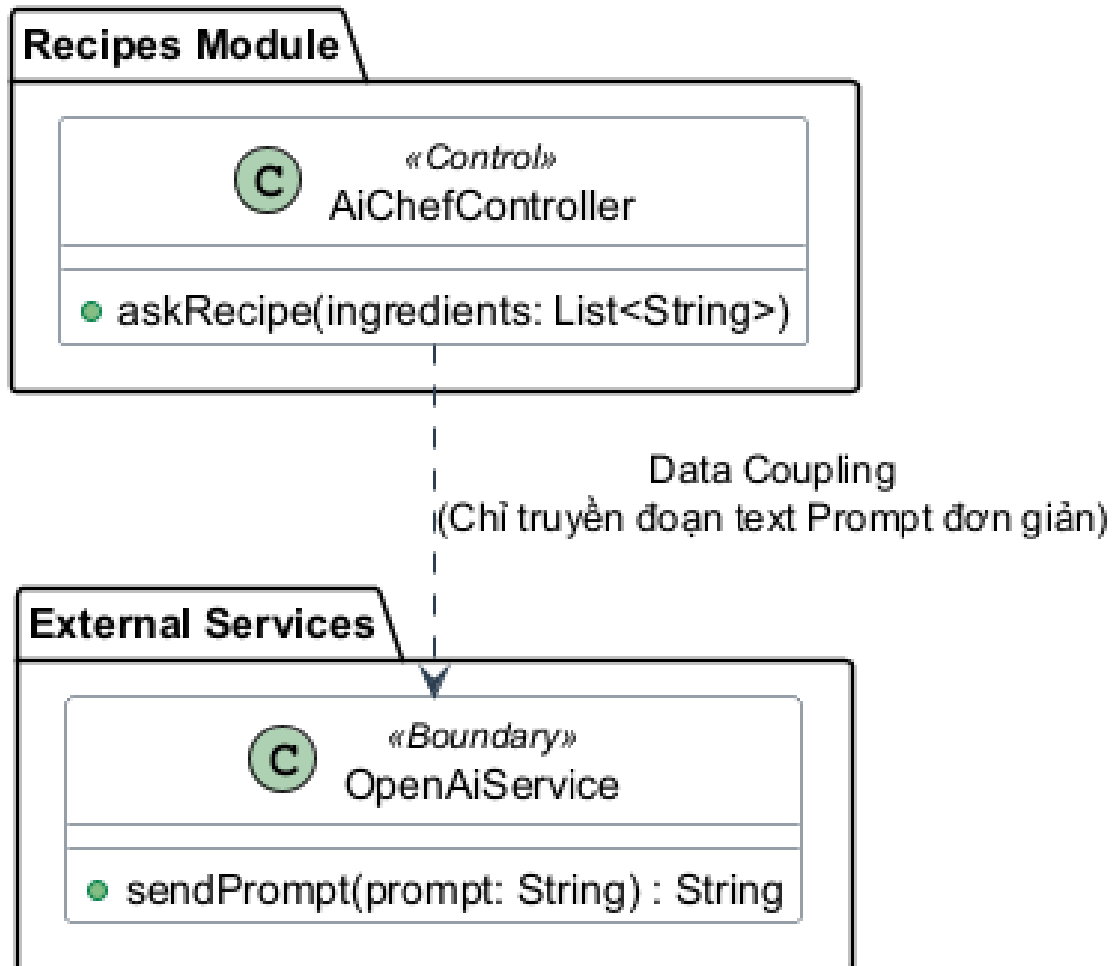
Module_5_Meals_Coupling_Diagram

Diagram A.13: Module_6_Recipes_Cohesion_Diagram

Functional Cohesion - Module 6 (Recipes & AI Chef)

Module_6_Recipes_Cohesion_Diagram

Diagram A.14: Module_6_Recipes_Coupling_Diagram

Data Coupling - Module 6 (Recipes & AI Chef)

Module_6_Recipes_Coupling_Diagram

Diagram A.15: Module_7_Reports_Cohesion_Diagram

Functional Cohesion - Module 7 (Reports)

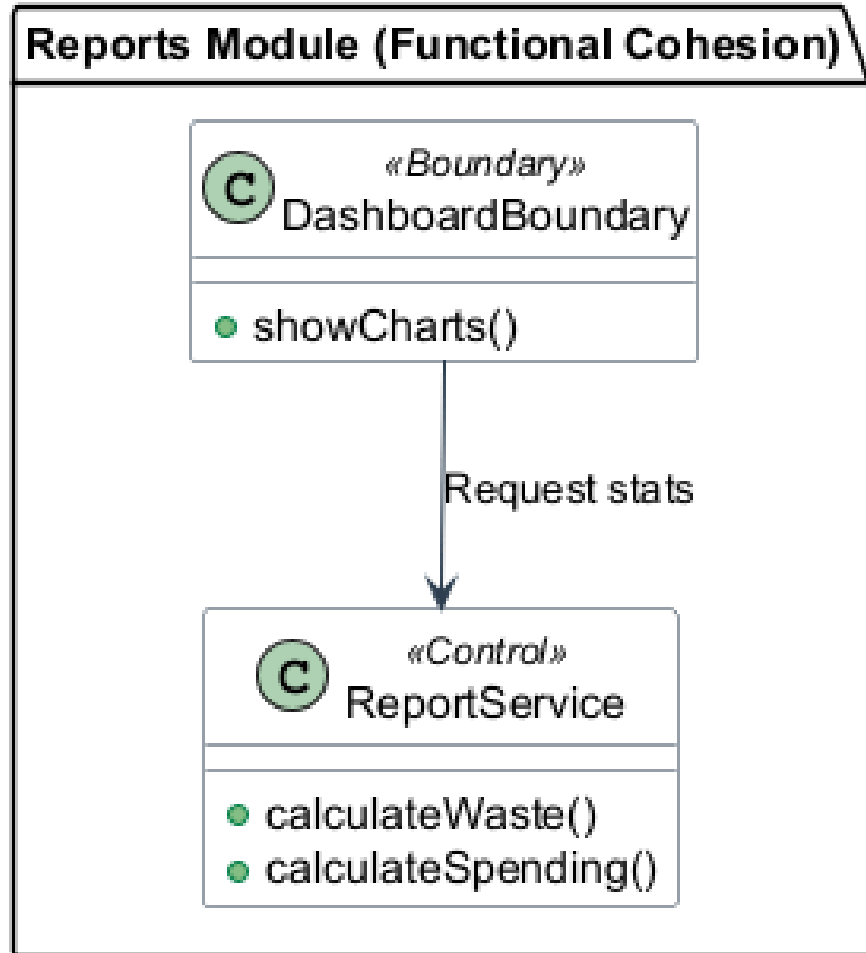
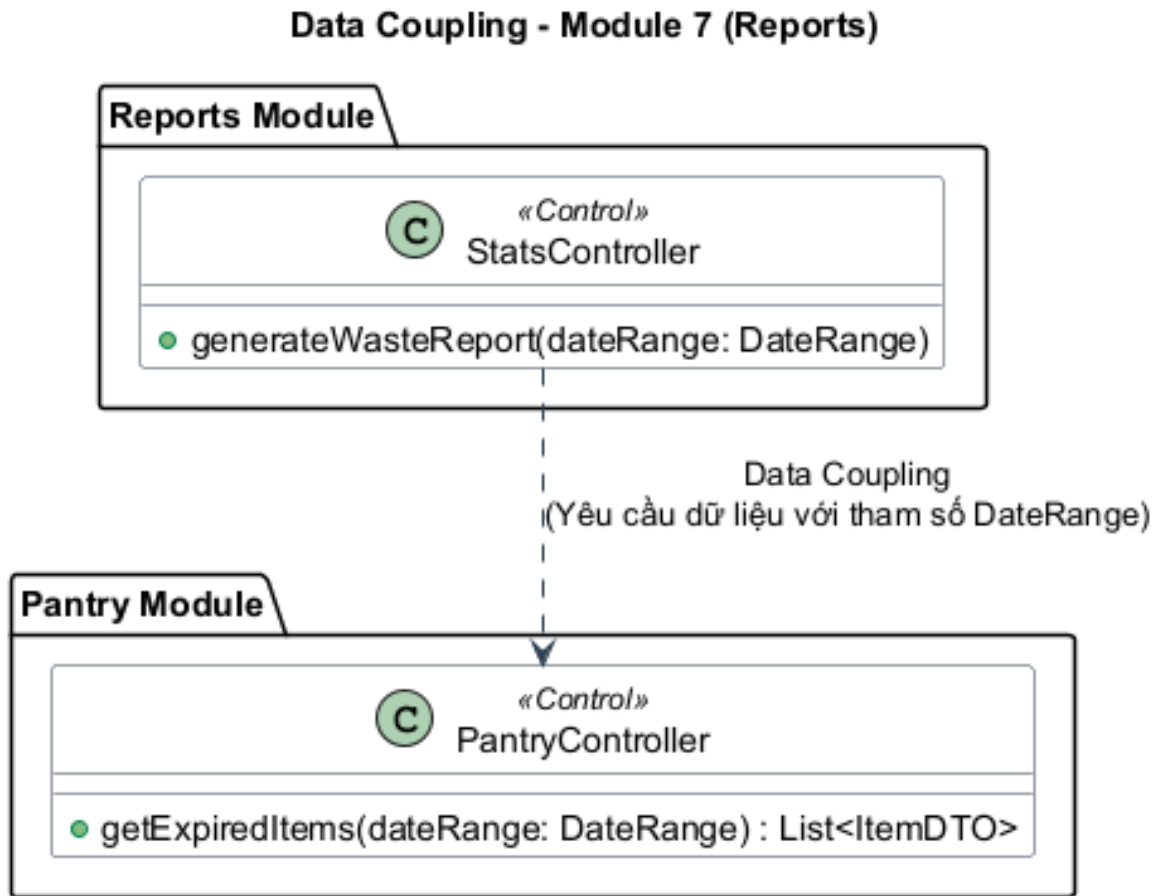
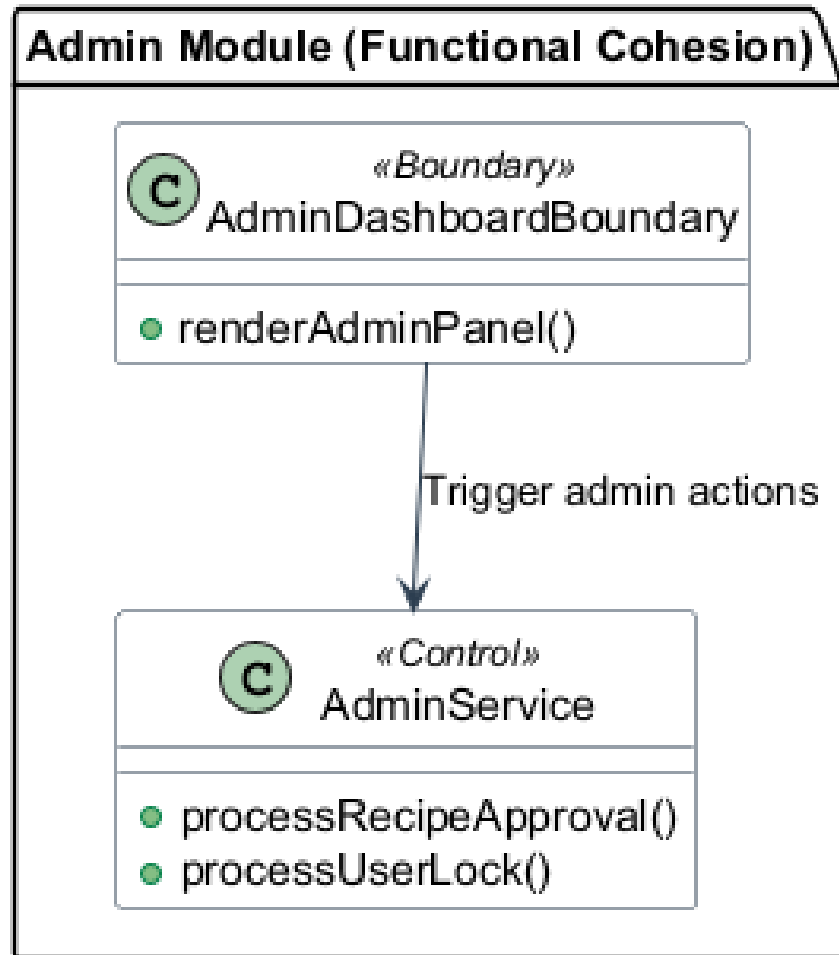
*Module_7_Reports_Cohesion_Diagram*

Diagram A.16: Module_7_Reports_Coupling_Diagram



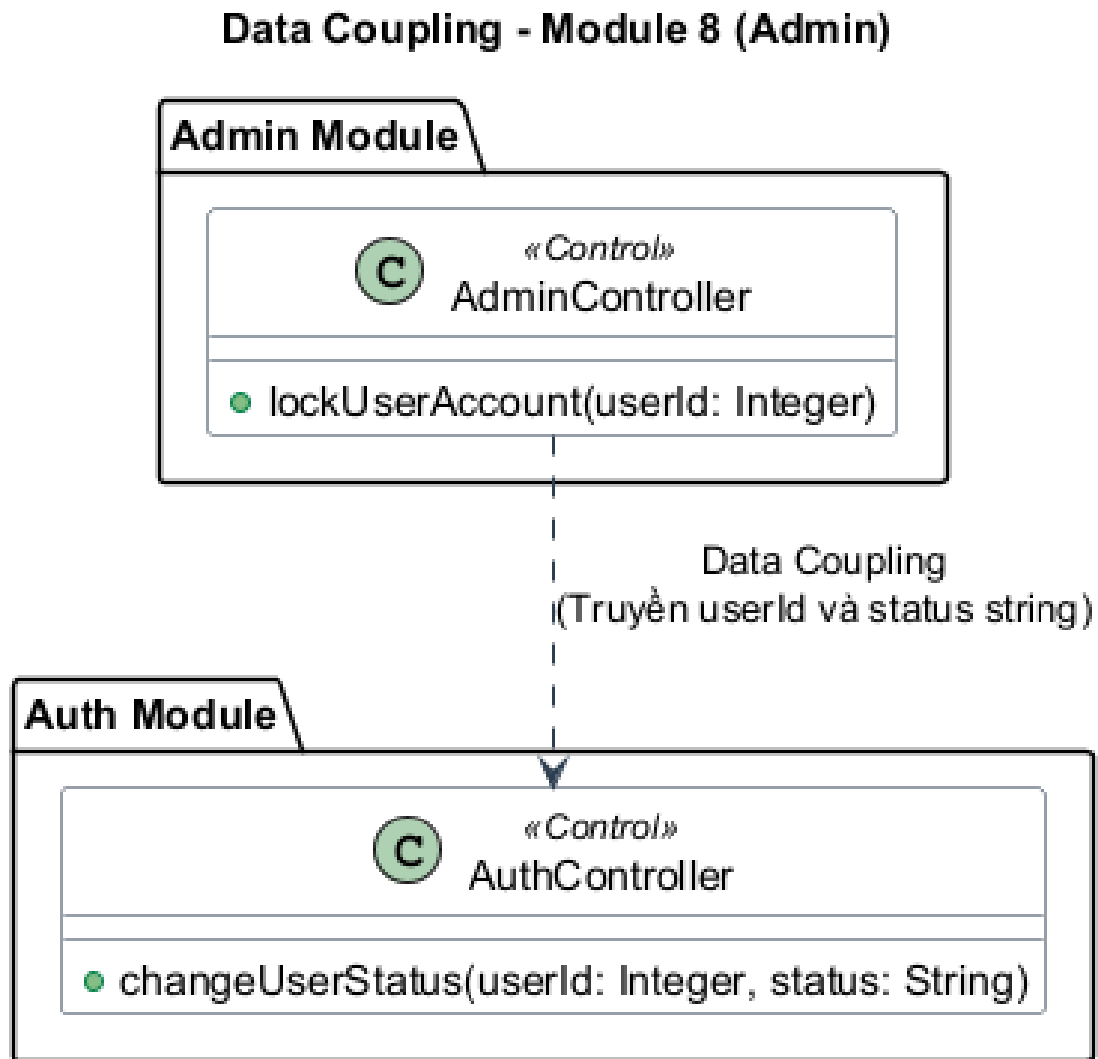
Module_7_Reports_Coupling_Diagram

Diagram A.17: Module_8_Admin_Cohesion_Diagram

Functional Cohesion - Module 8 (Admin)

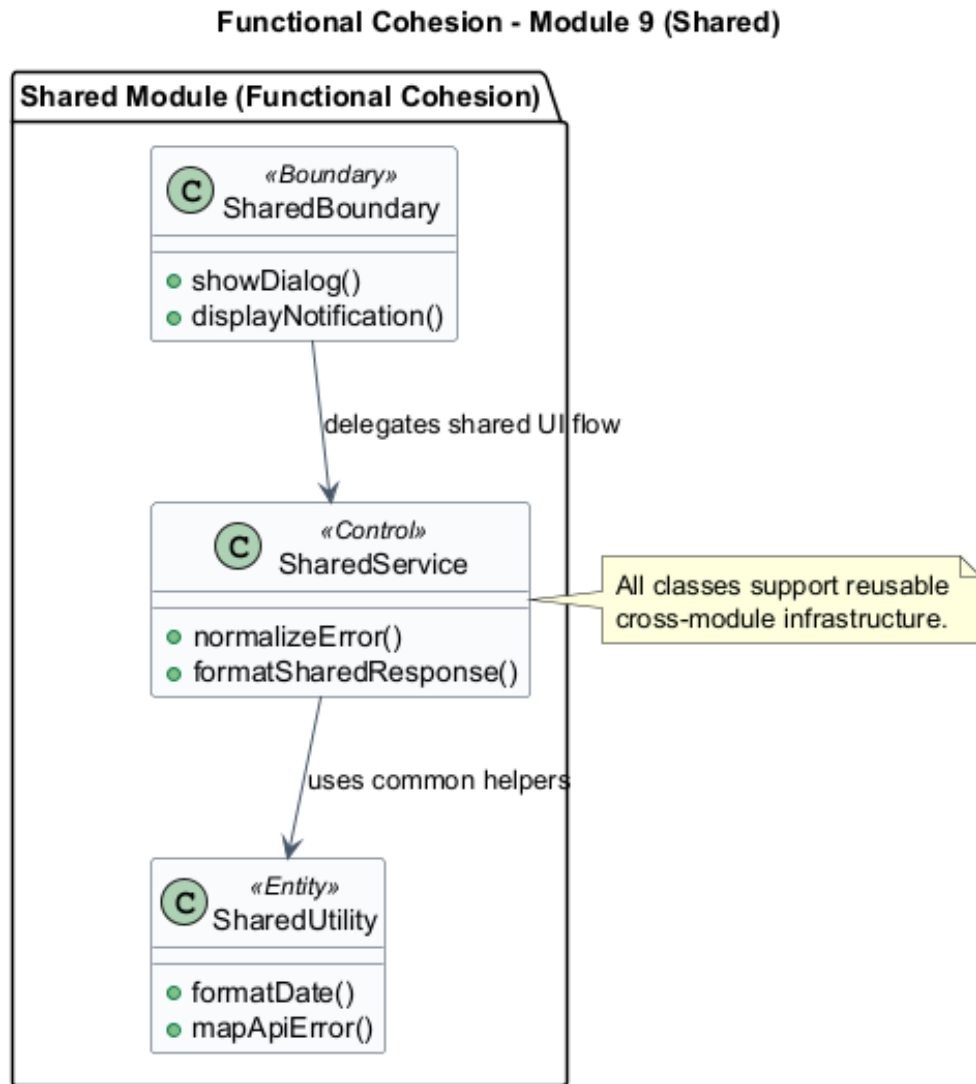
Module_8_Admin_Cohesion_Diagram

Diagram A.18: Module_8_Admin_Coupling_Diagram



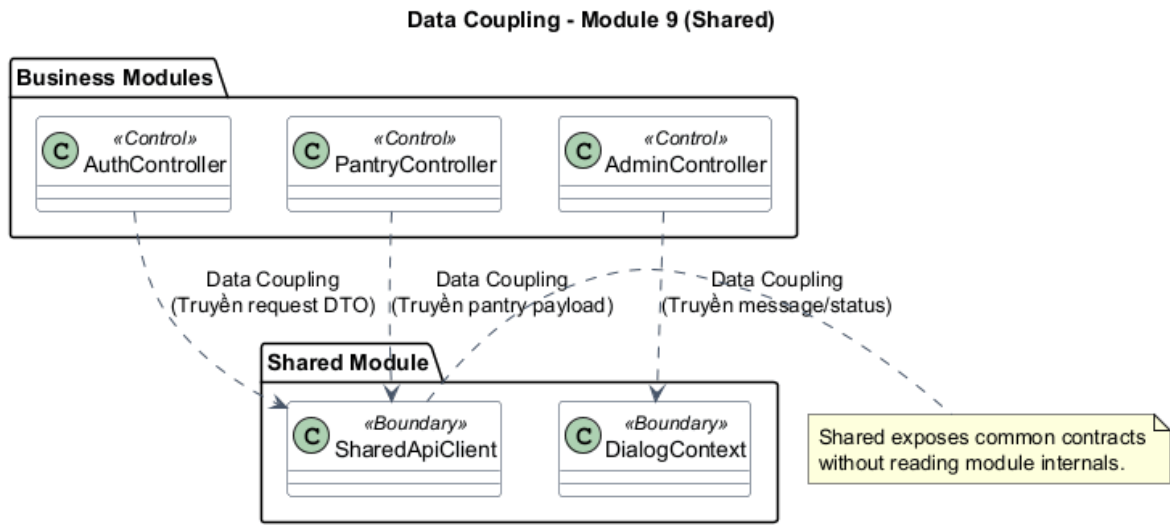
Module_8_Admin_Coupling_Diagram

Diagram A.19: Module_9_Shared_Cohesion_Diagram



Module_9_Shared_Cohesion_Diagram

Diagram A.20: Module_9_Shared_Coupling_Diagram



Module_9_Shared_Coupling_Diagram